UNIVERSITÀ DI PARMA

IoT Lab

→ LEC. 1

Internet of Things

Prof. Gianluigi Ferrari

Dipartimento di Ingegneria e Architettura – Pal. 2
Internet of Things (IoT) Lab
Università di Parma
gianluigi.ferrari@unipr.it
http://www.tlc.unipr.it/ferrari
http://iotlab. unipr.it/

Tel: 0521 90**6513**

Our Calendar (Spring 2026)

February						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
23	24	25	26	27	28	

March						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

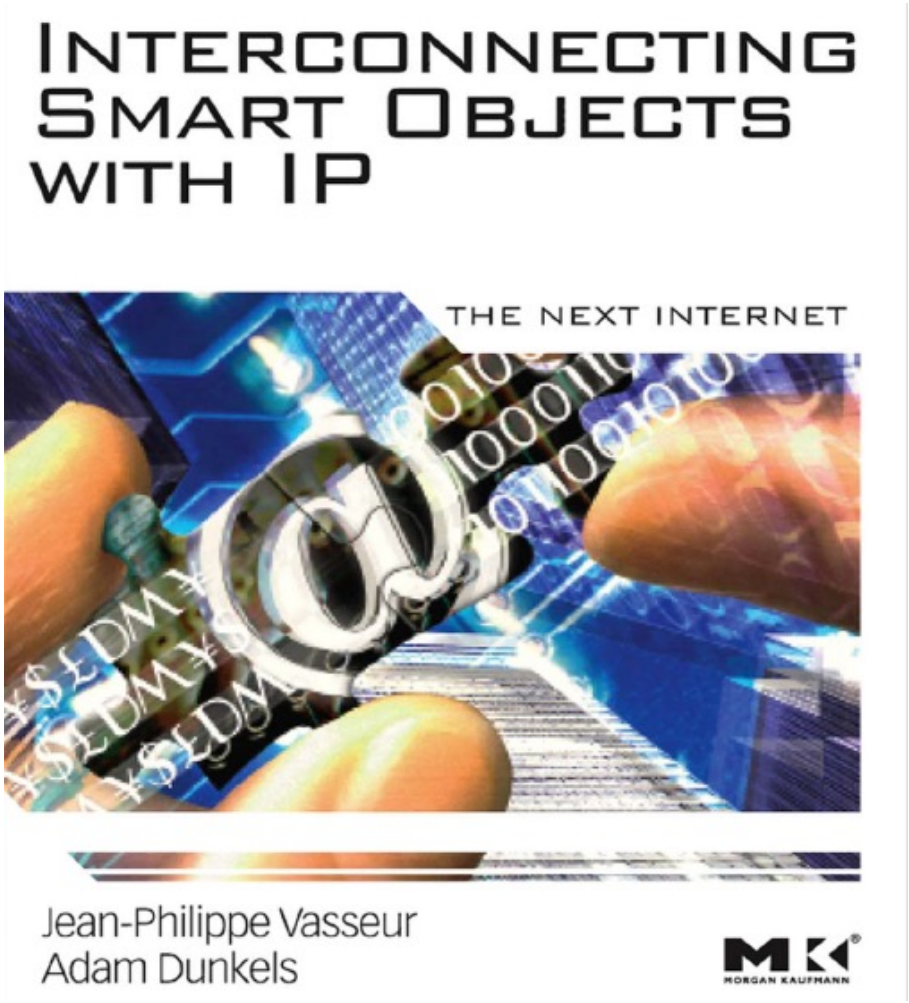
April						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

May						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
				1	2	3
4	5	6 -Midterm	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

June						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
1	2	3	4	5		

	theory
	practice
	no lecture
	Midterm (theory)

Our Main Reference



Outline

Theory

- Introduction to IoT
- Transport Layer Protocols: UDP and TCP
- IPv6
- Communication Mechanisms
- Energy-efficient MAC Protocols
- 6LowPAN
- Routing in LLNs & RPL
- Communication paradigms for the IoT
- Operating Systems and Hardware for the IoT
- Constrained Application Protocol (CoAP)
- IoT Security
- LoRa and LoRaWAN

Practice (Seminars, Lab & project)

- CoAP: Introduction to Java, Californium (Cf) Framework **or** Applied project (real devices)
- Lab & project assignment

UNIVERSITÀ DI PARMA

IoT Lab

Introduction to IoT

Main sources

- C. Bormann, “Protocols for the Real “Internet of Things””, keynote speech at the Internet of Things and Smart Cities Ph.D. School 2014 Lerici (SP), Italy - September 8th - 13th, 2014

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 1.

- M. R. Palattella; N. Accettura; X. Vilajosana; T. Watteyne; L. A. Grieco; G. Boggia; M. Dohler, "Standardized Protocol Stack for the Internet of (Important) Things," *IEEE Communications Surveys & Tutorials*, IEEE , vol.15, no.3, pp.1389,1406, Third Quarter 2013. doi: 10.1109/SURV.2012.111412.00158

Definition and Preliminaries

The Internet of Things (IoT)

- Extending the current Internet and providing connection, communication, and inter-networking between devices and physical objects, or "Things," is a growing trend that is often referred to as the *Internet of Things*.

- "The technologies and solutions that enable integration of real world data and services into the current information networking technologies are often described under the umbrella term of the Internet of Things (IoT)"

Other Definitions (1/2)

(1) The Internet of Things, also called The Internet of Objects, refers to a wireless network between objects, usually the network will be wireless and self-configuring, such as household appliances.

----- Wikipedia

(2) By embedding short-range mobile transceivers into a wide array of additional gadgets and everyday items, enabling new forms of communication between people and things, and between things themselves.

----- WSIS 2005

Other Definitions (2/2)

(3) The term "Internet of Things" has come to describe a number of technologies and research disciplines that enable the Internet to reach out into the real world of physical objects.

----- IoT 2008

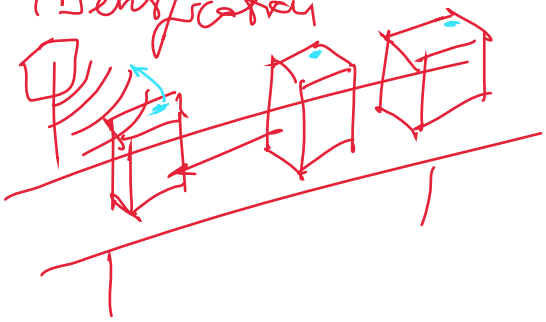
(4) “Things having identities and virtual personalities operating in smart spaces using intelligent interfaces to connect and communicate within social, environmental, and user contexts”.

------ IoT in 2020

SIoT = Social IoT

A brief History of IoT (RFID-oriented)

1997 "The Internet of Things" is the seventh in the series of ITU Internet Reports originally launched in 1997 under the title "Challenges to the Network" → RFID: bringing an object's identity to the Internet



1999 Auto-ID Center founded in MIT

2003 EPC Global founded in MIT

2005 Four important technologies of the internet of things was proposed in WSIS (World Summit on Information Society) conference.

2008 First international conference of internet of things: The IOT 2008 was held at Zurich.

Another History of IoT (Internet-oriented)

Year	Event
1967	REMBASS Remotely Monitored Battlefield Sensor System
1978	Distributed Sensor Networks for Aircraft Detection Lincoln Labs - Lacoss
1992	RAND Workshop - Future Technology Driven Revolutions in Military Conflict. Concepts behind Smart Dust emerge.
1993-1996	DARPA ISAT studies - many WSN ideas and applications discussed. Deborah Estrin leads one of the studies.
1994	LWIM - Low Power Wireless Integrated Microsensors - Bill Kaiser (UCLA)
1997	Smart Dust proposal written, Kris Pister (Berkeley)
1998	Seth Hollar makes wireless mouse collars
1999	Endeavour project proposed by Randy Katz, David Culler (Berkeley) PicoRadio project started by Jan Rabaey (Berkeley)
2000	Crossbow begins selling 'Berkeley motes'
2001	Multiple demos proving viability
2002	Dust, Ember, Millennial, Sensicast founded
2003	IEEE802.15.4-2003 standard Moteiv (now Sentilla) founded
2004	ZigBee 1.0 standard ratified TSMP 1.1 shipping
2005	Arch Rock founded
2006	ZigBee 2006 standard ratified IEEE802.15.4-2006 standard
2007	WirelessHART standard ratified IETF 6LoWPAN's RFC4944 published WirelessHART shown to achieve 99.999% reliability [20]
2008-2009	IETF workgroup Routing Over Low-power Lossy links (ROLL) created. IEEE802.15.4e work group created
2010-2011	IEEE802.15.4e's MAC protocol ratified IEFT 6LoWPAN's RFC4944 updated IETF ROLL's RPL routing protocol ratified

Internet of Things and Smart Cities Ph.D. School (2013,2014)

International speakers from academia and industry gave lectures tailoring their research field for an interdisciplinary audience. A dedicated discussion panel focused on the interaction and the collaboration between academia and industry in order to depict the future vision of Smart Cities and IoT

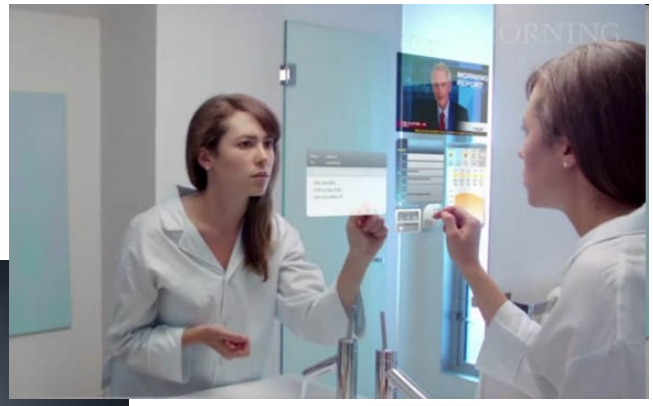
Some numbers:
36 students from 11 countries in Europe
17 speakers from Universities and Industry

2018 This Is What Happens In An Internet Minute
facebook
973,000 Logins
18 Million Text Messages
YouTube
4.3 Million Videos Viewed
375,000 Apps Downloaded
174,000 Scrolling Instagram
481,000 Tweets Sent
1.1 Million Swipes
tinder
187 Million Emails Sent
936,073 Views
twitch
67 Voice-First Devices Shipped
amazon echo
38 Million Messages
25,000 GIFs Sent via Messenger
2.4 Million Snaps Created
$862,823 Spent Online
NETFLIX
266,000 Hours Watched
3.7 Million Search Queries
Google
60 SECONDS
Created By:
@LoriLewis
@OfficiallyChadd

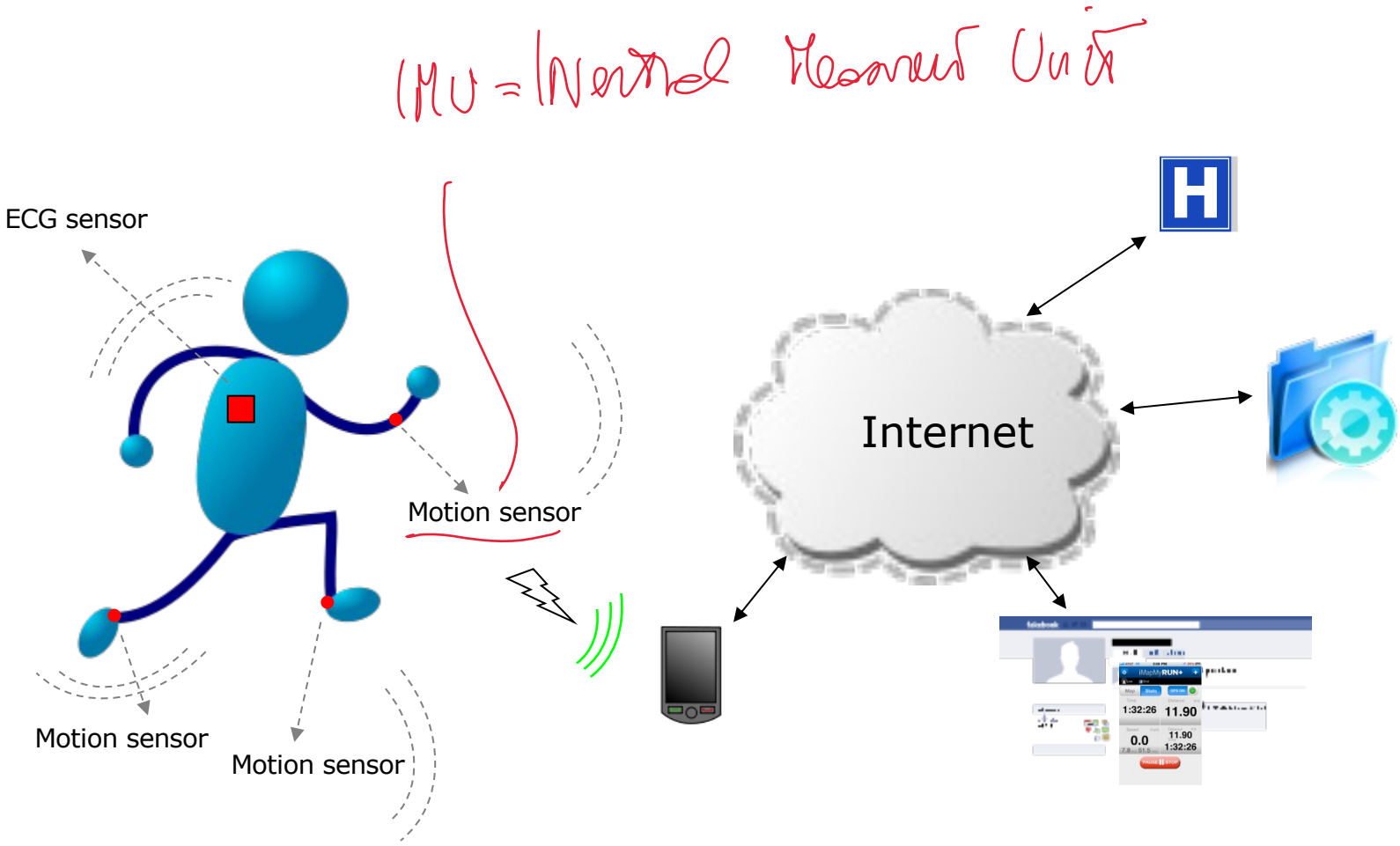
2019 This Is What Happens In An Internet Minute
facebook
1 Million Logging In
18.1 Million Texts Sent
YouTube
4.5 Million Videos Viewed
390,030 Apps Downloaded
347,222 Scrolling Instagram
87,500 People Tweeting
1.4 Million Swipes
tinder
188 Million Emails Sent
1 Million Views
twitch
41 Music Streaming Subscriptions
180 Smart Speakers Shipped
amazon echo
4.8 Million Gifs Served
GIPHY
41.6 Million Messages Sent
2.1 Million Snaps Created
$996,956 Spent Online
694,444 Hours Watched
NETFLIX
3.8 Million Search Queries
Google
60 SECONDS
Created By:
@LoriLewis
@OfficiallyChadd

More “Things” are being connected

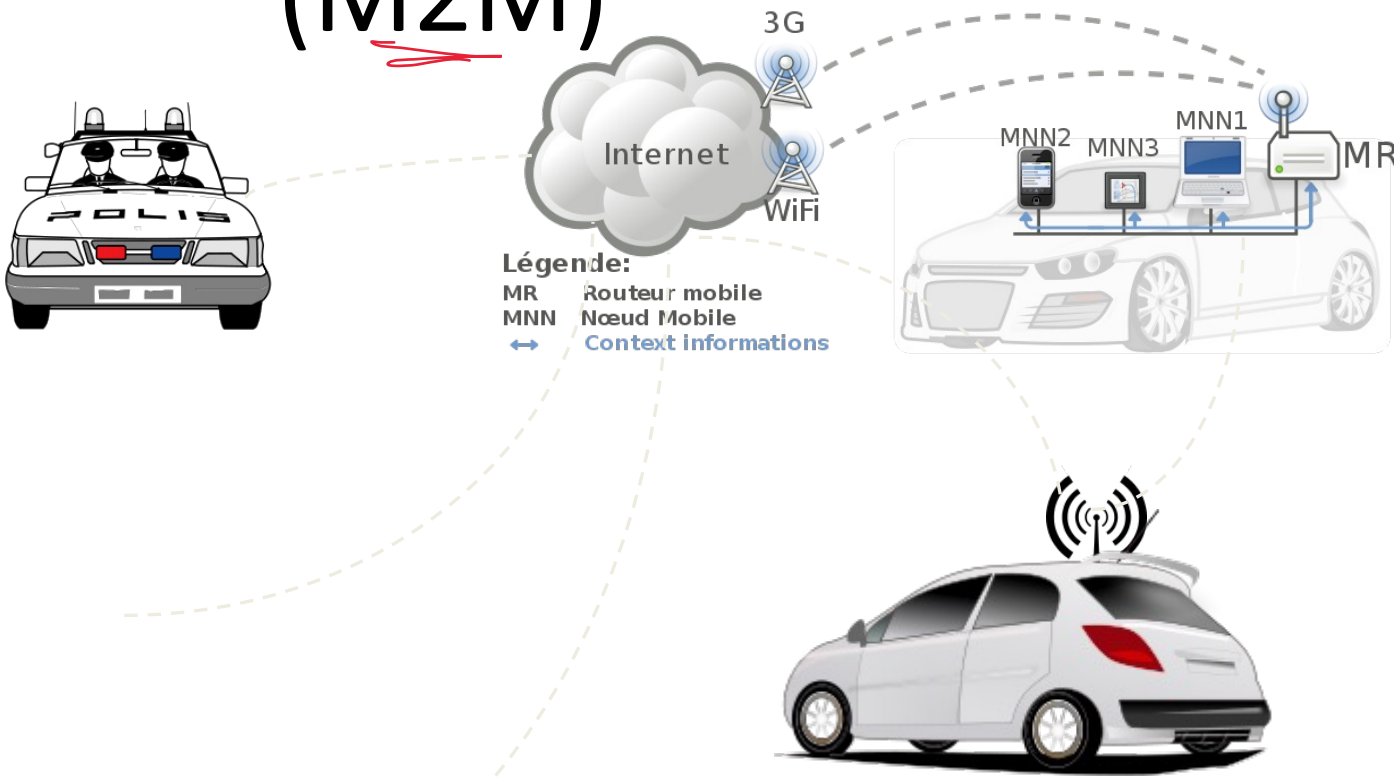
Home/daily-life devices
Business and
Public infrastructure
Health-care
…



People Connecting to Things



Things Connecting to Things (M2M)

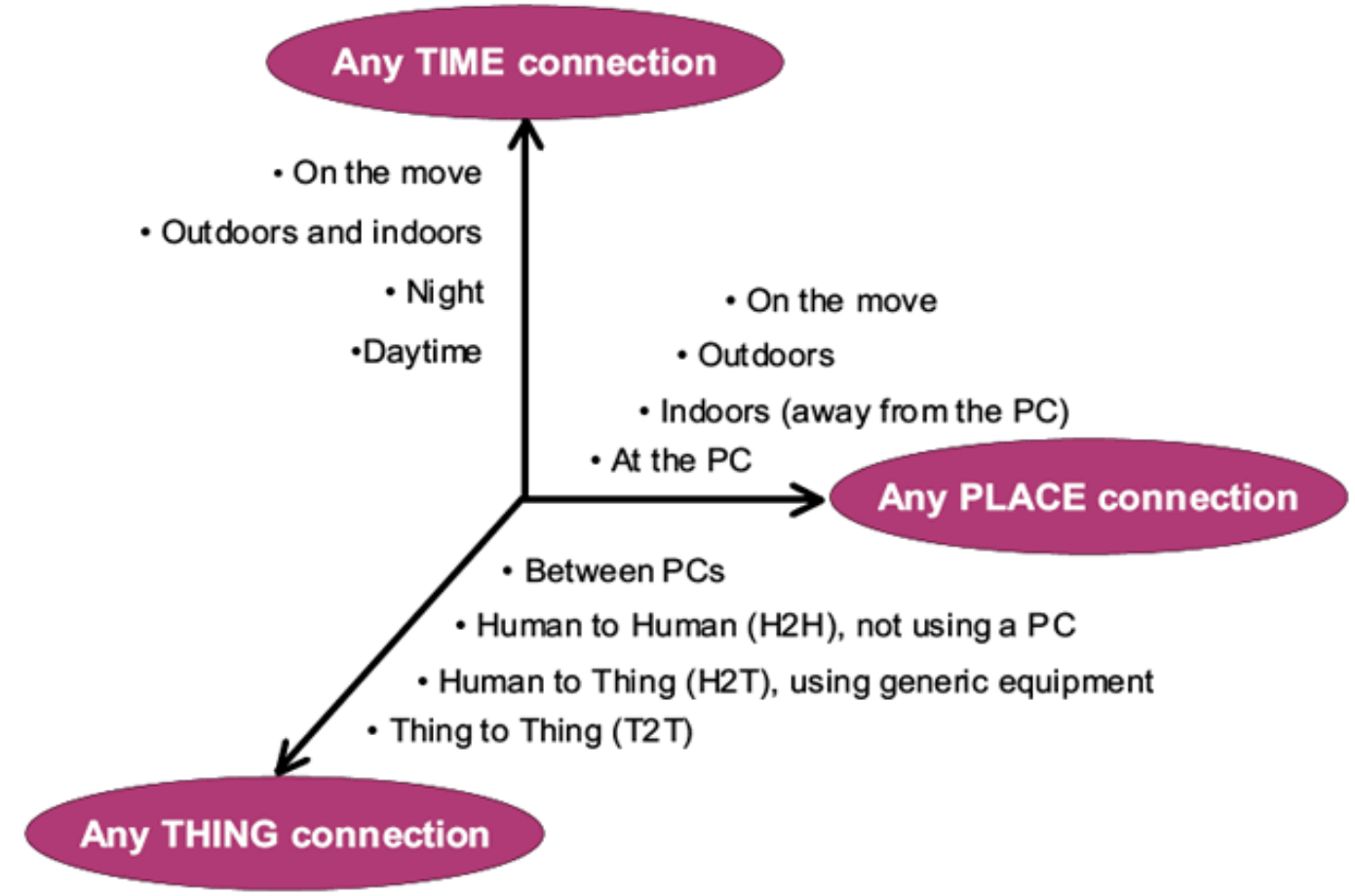


- Complex and heterogeneous resources and networks

How are the networks changing?

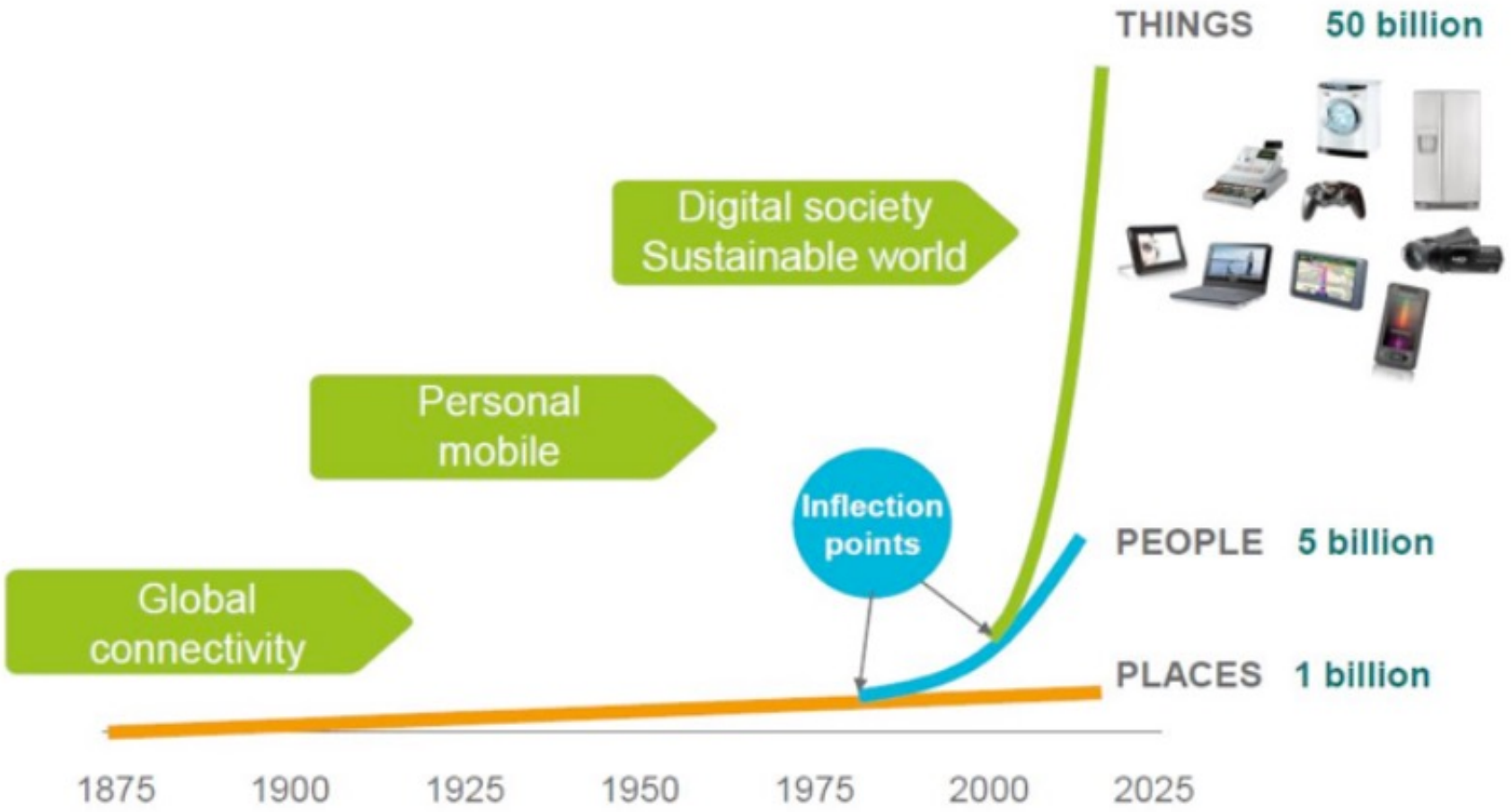
- Extensions
  - More nodes, more connections, IPv6, 6LowPan,...
  - Any TIME, Any PLACE + Any THING
  - M2M, IoT
    - Billions of interconnected devices,
    - Everybody connected.
- Expansions
  - Broadband
- Enhancements
  - Smart networks
  - Data-centric and content-oriented networking
  - Context-aware (autonomous) systems

Future Networks



Source: ITU adapted from Nomura Research Institute

Connecting:
Places → People → Things



A New Perspective

- Scale up:
  - Number of nodes (50 billion by 2020)
- Scale down:
  - Node
  - Cost
  - Complexity
  - Reuse: cent, kylobyte, megahertz

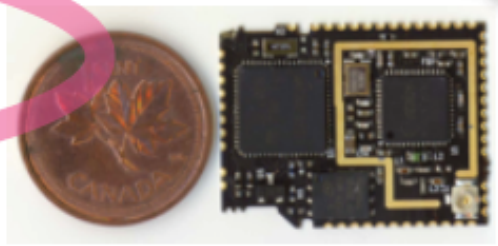
Constrained nodes: orders of magnitude

10/100 vs. 50/250

- There is not just a single class of "constrained node"

RFC 7228

- Class 0: too small to securely run on the Internet
  - "too constrained"
- Class 1: ~10 KiB data, ~100 KiB code
  - "quite constrained", "10/100"
- Class 2: ~50 KiB data, ~250 KiB code
  - "not so constrained", "50/250"

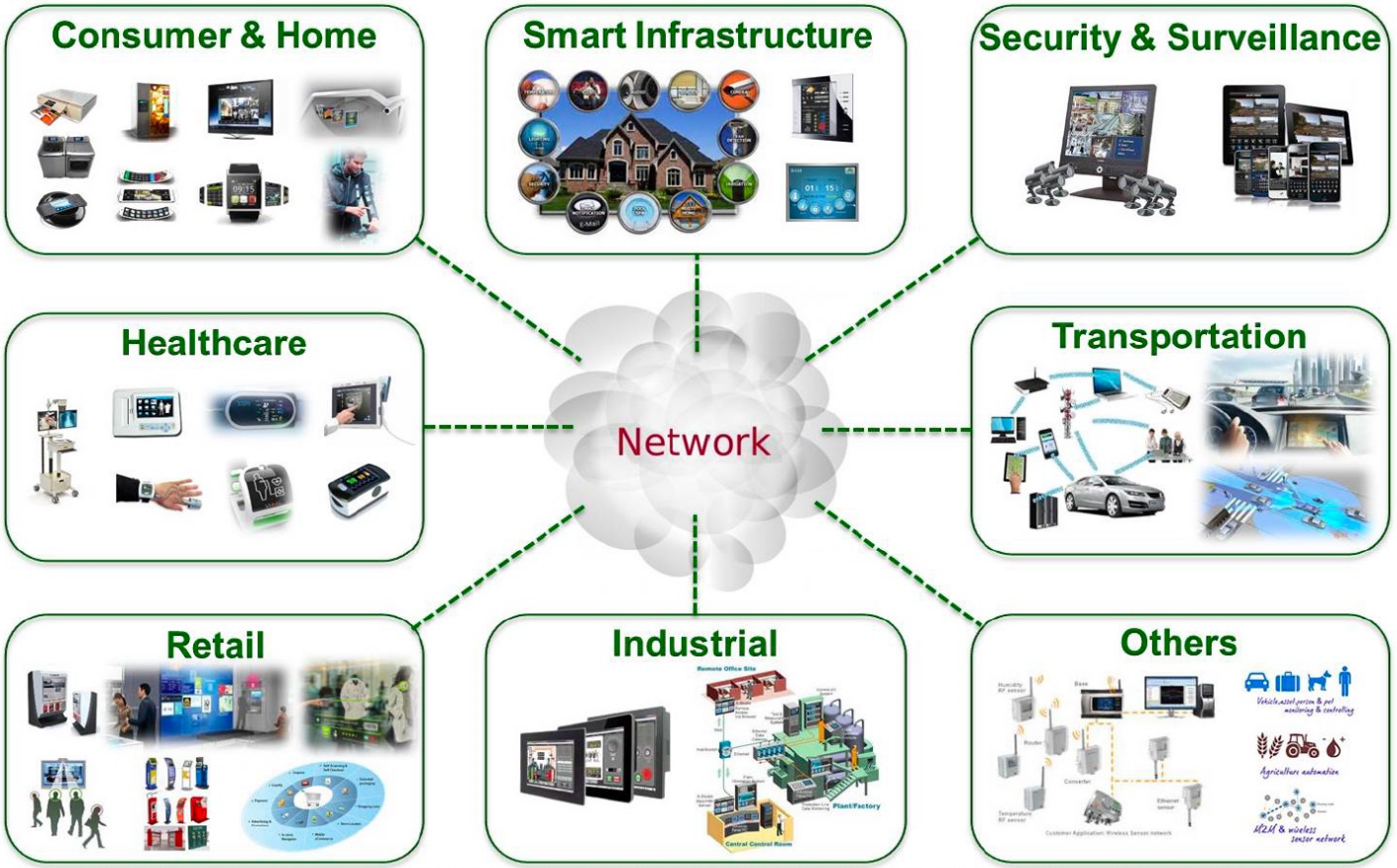


- These classes are not clear-cut, but may structure the discussion and help avoid talking at cross-purposes

Smart agriculture

EXPO 2015: UK Pavillion
Accelerometers to understand the status of a beehive

IoT Applications



Vivante and the Vivante logo are trademarks of Vivante Corporation. All other product, image or service names in this presentation are the property of their respective owners. © 2013 Vivante Corporation

Today's Smart Objects (1)

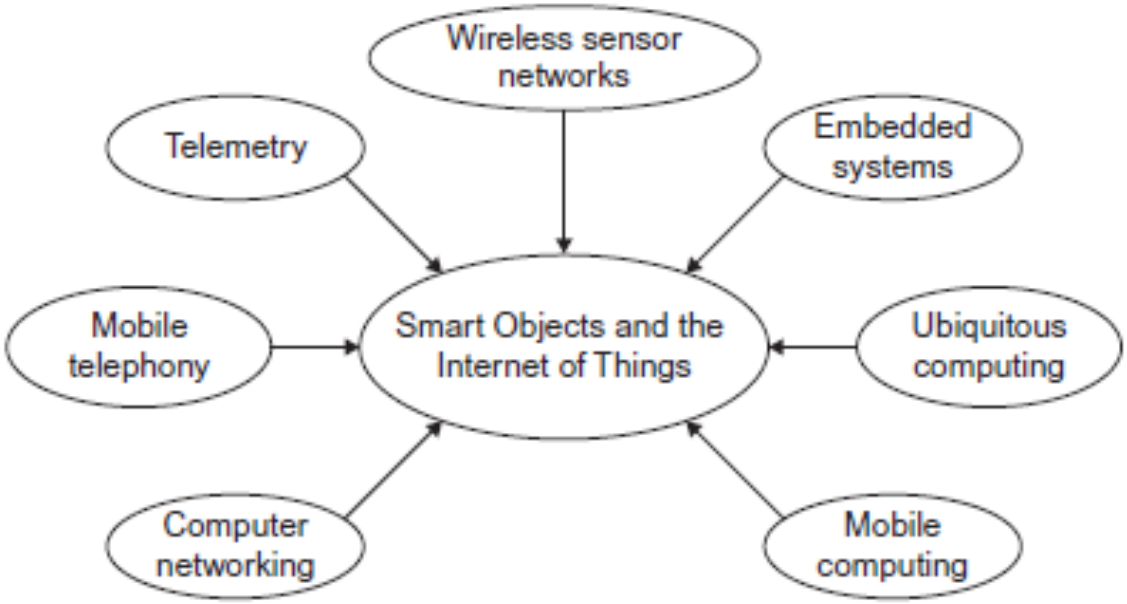


FIGURE 1.1

Smart objects are the intersection of embedded systems, ubiquitous computing, mobile telephony, telemetry, wireless sensor networks, mobile computing, and computer networking.

Embedded Systems (1)

- A computer (microprocessor) embedded in something other than a computer
- Traditionally: real-time systems (strict time response requirements, industry) → Real-Time Operating System (RTOS)
- Commonalities (emb. sys. - smart obj.):
  - Hardware
  - Constraints in terms of computational power and memory
  - Same microcontrollers (ARM, RISC-V)
- Differences:
  - Communication is not core for embedded systems (e.g., a car engine which cannot communicate can still work as car engine)
  - Communication is crucial for a smart object

Embedded Systems (2)

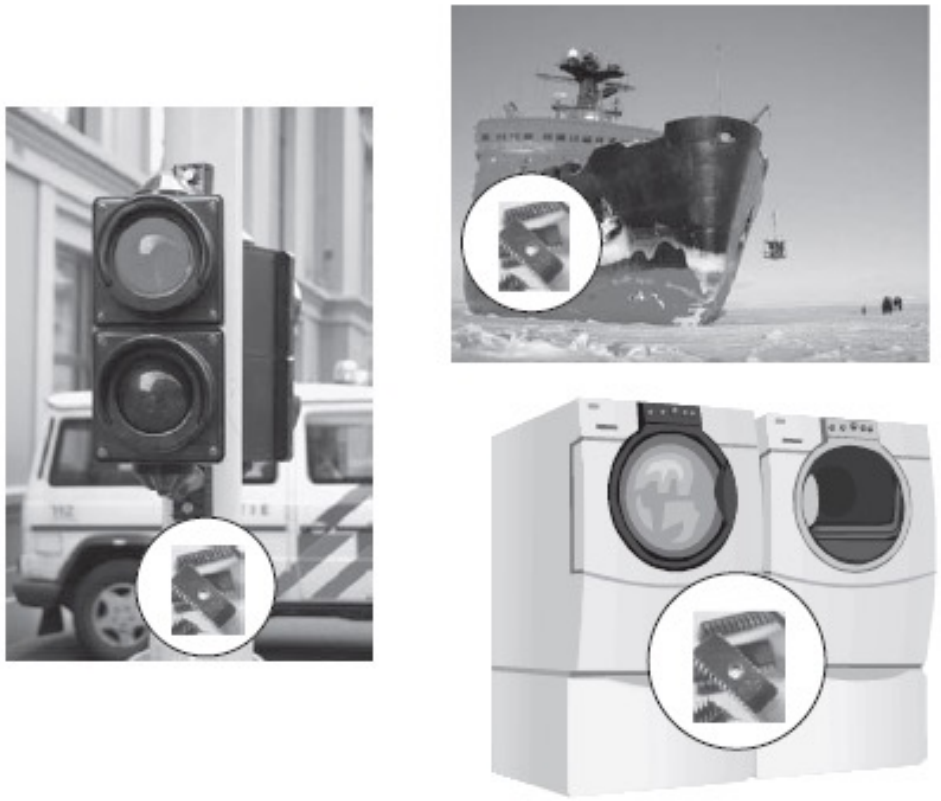


FIGURE 1.2

Embedded systems are microprocessor-equipped systems and devices that interact with the physical world. Examples include traffic lights, a ship's rudder controllers, and washing machine controllers.

Ubiquitous and Pervasive Computing (1/2)

- Field of study based on concept of what happens when computers move away from the desktop and immerse in the environment (MIT, 1988): "computing will no longer be done by desktop pcs, but will move into our daily environment"
- Ubiquitous computing: successful measure is its invisibility
- Examples:
  - Active badges (AT&T@ Cambrige, UK, late 80's-early 90's) with readers
  - AmbientROOM project (MIT, 1996)
  - Wearable computing: computing infrastructure moves on the body (**our research**)

Ubiquitous and Pervasive Computing (2/2)

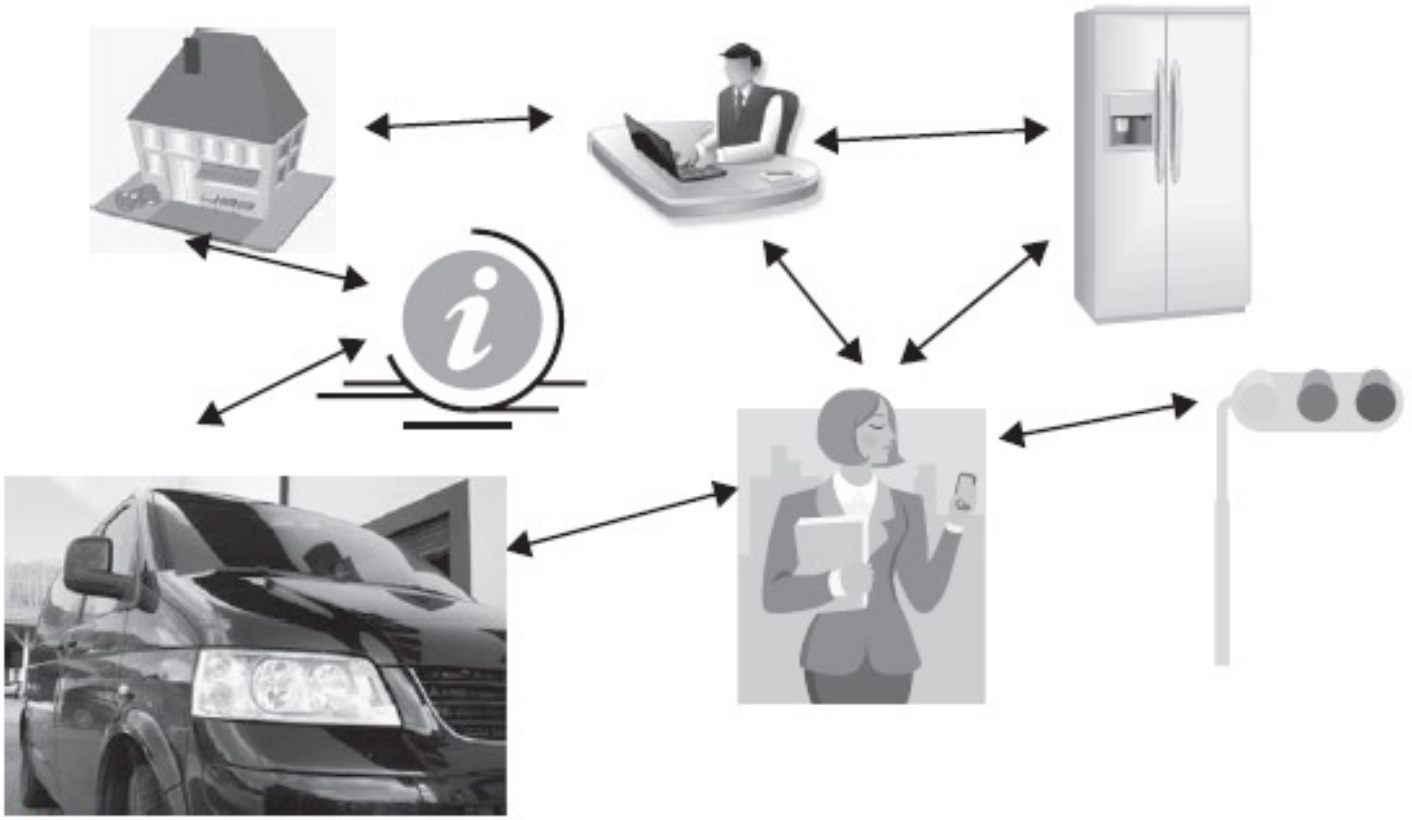


FIGURE 1.3

Ubiquitous computing is a vision for the future of computers where computing moves into everyday objects.

Mobile Telephony

- Grew from telephony industry with the promise of ubiquitous access to telephony
- Typically thought of as cellular telephony
- Smartphones: far more than cellular phones
  - Several other wireless interfaces: WiFi, Bluetooth, NFC
  - A large number of on-board sensors
- Mobile telephony has revolutionized access to the Internet → really seamless
- So, smartphones are likely to play a key role in IoT

Telemetry and M2M

- Telemetry: remote measurements
- M2M: generalization of telemetry, which implies autonomic communication between non-human operated machines and central to the concept of telemetry
- Telemetry:
  - Long-range: temperature, humidity, wind from remote weather stations (cellular network or Inmarsat satellite network); Formula 1
  - Short-range: pacemaker (telemetry coil, to measure the activity without surgery: low-power communication)

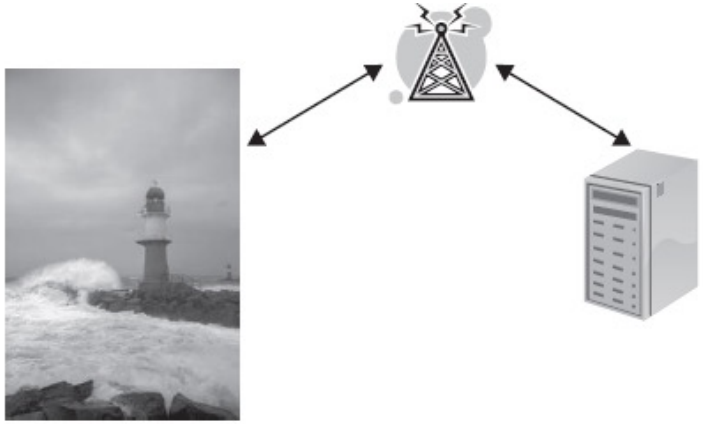


FIGURE 1.4

Telemetry allows reading measurements from remote systems such as weather stations. Data are typically transported using mobile telephony systems.

Wireless Sensor Networks

- Derive from the idea that wireless sensors can collect information from the physical world in a large number of situations (wild fire tracking, agriculture management, etc.)
- Significant research activity in the 2000's:
  - Smart Dust: Berkeley, 1999
  - First prototypes with larger sizes
- Idea: try to be different from the Internet → cross-layer optimization → no scalability
- Several results (energy savings, algorithms, protocols) have been moved to smart objects (not only data gathering, but actuation/control)

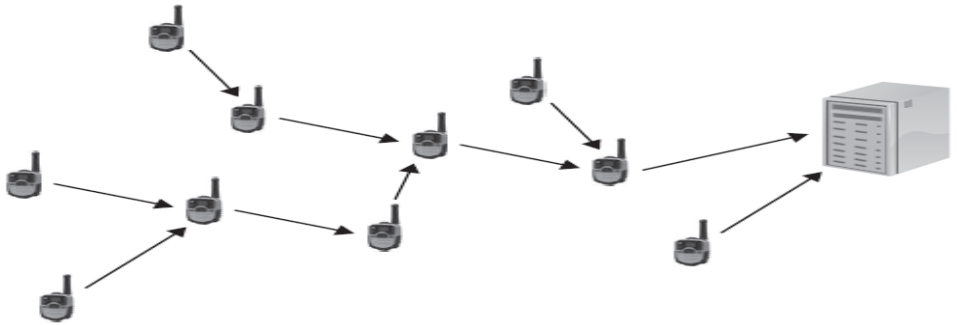
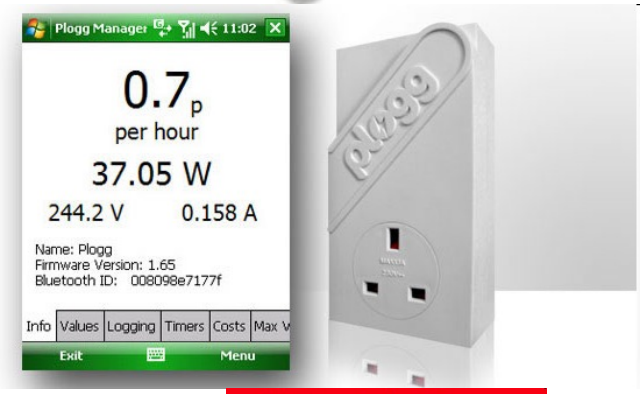
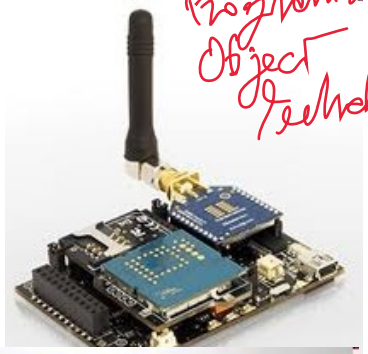


FIGURE 1.5
Wireless sensor networks provide large-scale measurements of physical properties using large amounts of sensors that transport their data wirelessly to a base station.

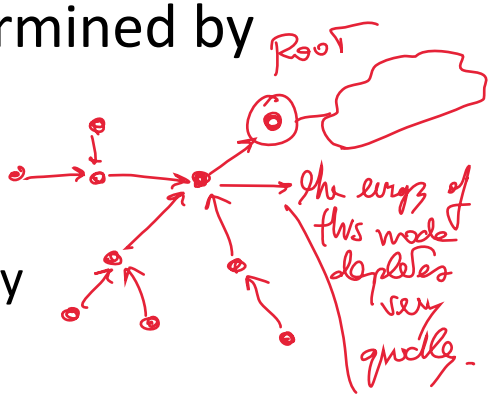
Sensor devices are becoming widely available

- Programmable devices
- Off-the-shelf gadgets/tools





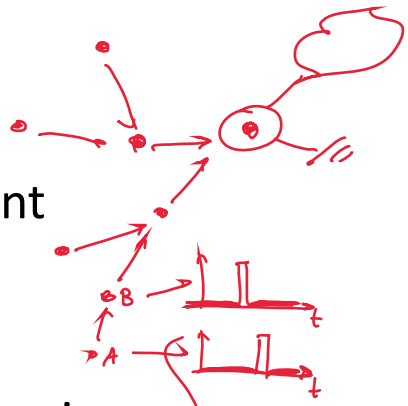
Challenges for Smart Objects



- **Node-level** internals: power consumption and physical size
- **Network-level** mechanisms and structures: determined by the entire ensemble of smart objects
- Aspects are highly interrelated:
  - power consumption (node-level) is highly influenced by communication patterns (network-level);
  - a routing protocol (network-level) should take into account the energy statuses of the nodes (node-level)
- Internet Protocol for Smart Objects (IPSO) Alliance: set up to spread awareness of the technology around smart objects → smart objects need evolvable technology → technology around IP is the right one

Node-level Challenge: Power Consumption

- Power consumption: battery-power or external low-power energy source (physical vibrations)
- Design implications for hardware, software, network protocols and architecture
  - Hardware designer: low-power components, minimal current leakage, power-efficient sleep mode
  - Software designer: make the most out of hardware
  - Operating systems for smart objects: natively provide mechanisms for low-power operation
- Power efficiency significantly affects network architectures and protocols design
  - Communication patterns→ use efficiently resources (sleeping)
  - Hardware and software keep track of energy spent and communicate to network layer



Node-level Challenges: Physical Size and Cost

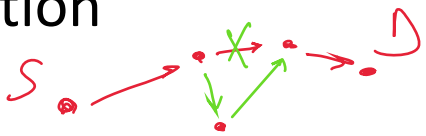
- Physical size: size and form factor determine the potential applications for a given smart object system
- Hardware designer
  - Hardware small
  - Number of components small
  - Each component small and inexpensive
- Software designer
  - Microprocessor smaller → Reduced computational speed and memory size
  - Each "line of code" (protocol "feature") counts: a few kBytes (e.g., efficient multicast in 6LoWPANs)
- Network protocols: designed minimizing the amount of information a node keeps about the network and other nodes
- Memory constraints: network level ←→ node level

Network-level Challenge: Network Size

- Large networks of small objects (e.g., thousands of nodes) → large amount of generated data
  - Example: Smart object network that samples the temperature inside a building. Temperature is generally a slow-moving phenomenon, so the nodes do not need to sample very often: sample the temperature twice every minute → 737,280 readings per year. If the system is designed to work for ten years, there will be over seven million readings, from each node, during the lifetime of the system
- Network size → impact on routing protocol design
- Routing protocol is related to:
  - amount of data the network can sustain
  - speed at which these data can be successfully transported through the network
  - achievable lifetime of the network as a whole
- Routing protocol must make well-informed choices: information on the network as a whole and on the neighbors

Network-level Challenge: Lossy Communications

- Unreliable nature of smart object networks
- Lossyness: inherent property of smart object networks → design for lossy networks, so that the communication protocol is robust for both lossy and non-lossy



- Lossy links → impact on routing protocol design
- Lossyness: illusive property, particularly in wireless networks.
  - Affected by environmental factors (e.g., temperature and humidity)
  - Electromagnetic interference
- Routing protocols for smart object networks must be prepared for this illusiveness.

↳ Need to have automatic mechanisms.

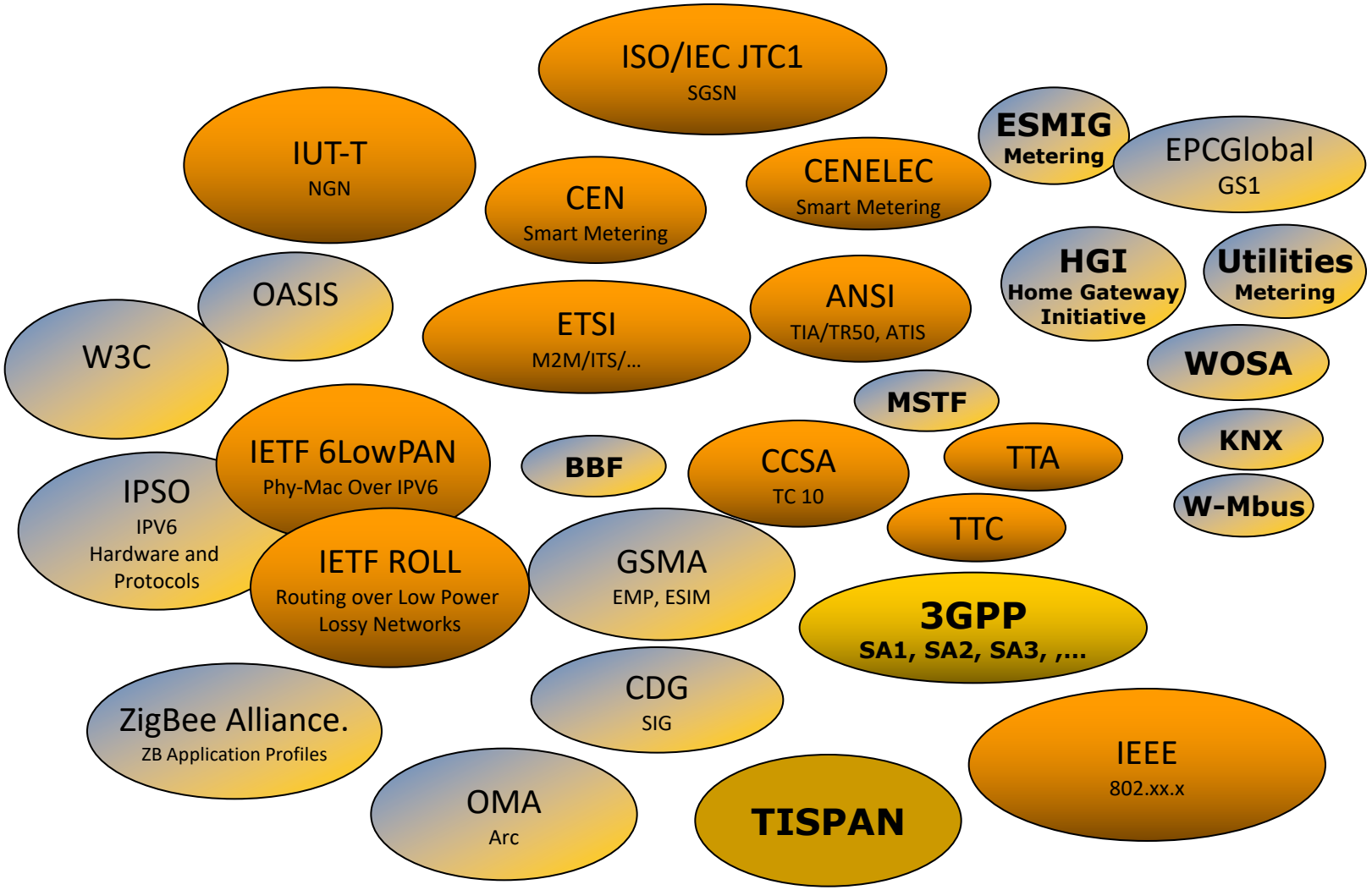
Other Network-level Challenges

- Node addressing: each smart object must be addressable uniquely
  - Within a single (even if large) network
  - With virtually any other SO of any other network → exponential growth → IPv6
- Network management: very challenging (no manual intervention; self-organization)
- Scale of smart object networks: amount of different environments and types of systems in which smart objects are used → various communication technologies
- Smart object network: provide mechanisms for external access to itself

Other relevant aspects: Standardization

- Different technology manufacturers have different specializations
  - For example, environment control system is controlled by an advanced IT system that receives its input from the humidity sensors
- With standardized technology, the technology is independent of its vendors, producers, and users → high acceptance
- Open standards produced through established practices of international standardization organizations
  - Patents for technology standardized must be freely accessible (protection against "submarine" patents)
- Standardization of smart object technology → challenge with several organization (different levels of protocol stack)

Standards Development Organizations (SDOs) and Industrial Groups



M2M: Vertical Fragmentation

- **Fragmented industry verticals:**
  - most industries are solving their M2M needs on their own;
  - often converging upon similar architectures and elements;
  - the M2M "wheel" is reinvented from industry to industry.
- **Customized solutions abound:**
  - since solutions are on an industry-by-industry basis, each solution requires its own design, production, and implementation cycle;
  - unique solutions often result in sole-sourcing of customized hardware;
  - software design is limited to a small group of developers who understand the architecture and API's for a particular industry or platform, thus leading to high development costs and high costs for support.
- **Lack of scale for components:**
  - economies of scale are difficult to achieve in any given industry since solutions are so varied and unique.

Examples of Verticals (1/2)

Security	Surveillance applications, alarms, object / people tracking
Transportation	Fleet management, emission control, toll payment, road safety (e.g. eCall stow pipe)
Health care	e-Health, and personal security
Utilities	Smart Metering and Grid such as oil, water, electricity, heat, and others
Manufacturing	Production chain monitoring and automation

Examples of Verticals (2/2)

Supply and provisioning	Freight supply and distribution monitoring, Vending Machines
Facility Management	Home / building / campus automation

. . .

Future	Multitude of new M2M applications based on standardized Service Capabilities and API [Ericsson CEO vision of 50 Billion Connected Objects by 2020]

Industrial Standardization for M2M

- M2M is here, and has massive potential for growth to be unleashed by standards.
- M2M first step towards future Internet of things (IoT) and will support Smart Metering/Grids.
- Market value increases with the “square” of interoperable/ accessible services.
- Operators and vendors see ETSI as a good place to do the work, however, progress requires wider consensus. (IETF, OMA and BBF contributes parts)

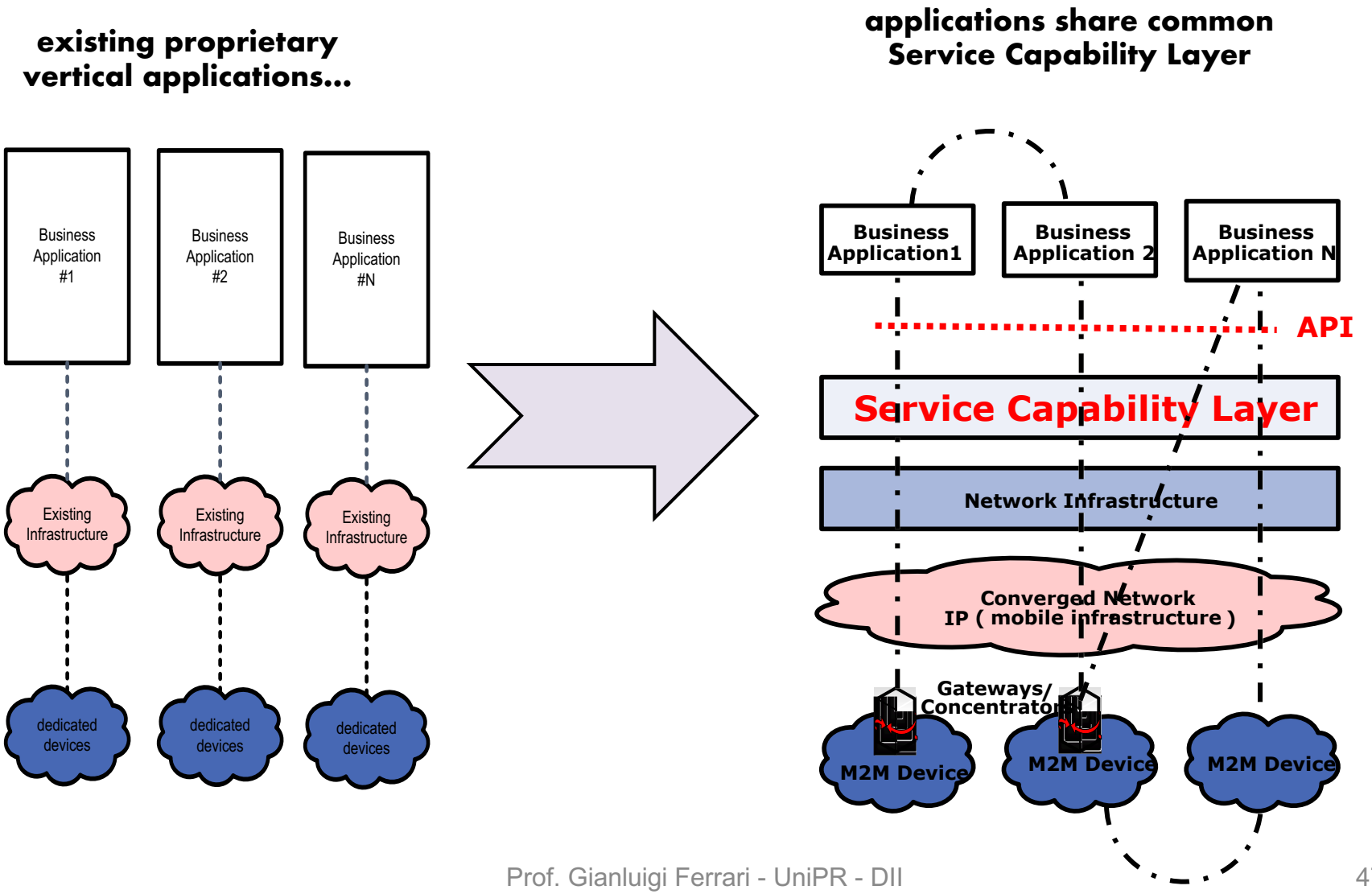
ITU-T M2M Service Layer Focus Group established in Jan. 2012

- M2M is included under the larger umbrella of the "Internet of Things" (IoT).
- The group will study and evaluate the M2M landscape and M2M work currently being undertaken by regional and national standards development organizations (SDOs), with a view to identifying a common set of requirements.
- The Focus Group will initially focus on the APIs and protocols to support e-health applications and services.
- The Focus Group will work closely with all ITU-T Study Groups and with other relevant SDOs, forums/consortia, regulators, policy makers, industry and academia. Within the ITU, the group will work particularly closely with the Internet of Things Global Standards Initiative (IoT-GSI). http://www.itu.int/en/ITU-T/gsi/iot/Pages/default.aspx.

Merits of Service Capabilities and API

- Service Capabilities provide functions that are to be shared by different applications in an interoperable manner.
- Service Capabilities expose functionalities through open interfaces (e.g. an API).
  - This will support interoperable ubiquitous services, however, area-specific data-models may be required (e.g. for Smart Metering /AMR).
- Service Capabilities simplify and optimize applications development and deployment and hide technology specifics.
- Service Capabilities may be M2M specific or generic, i.e. providing support to other than M2M applications.

Needs to invert the stove pipes



IETF (Academy-friendly)

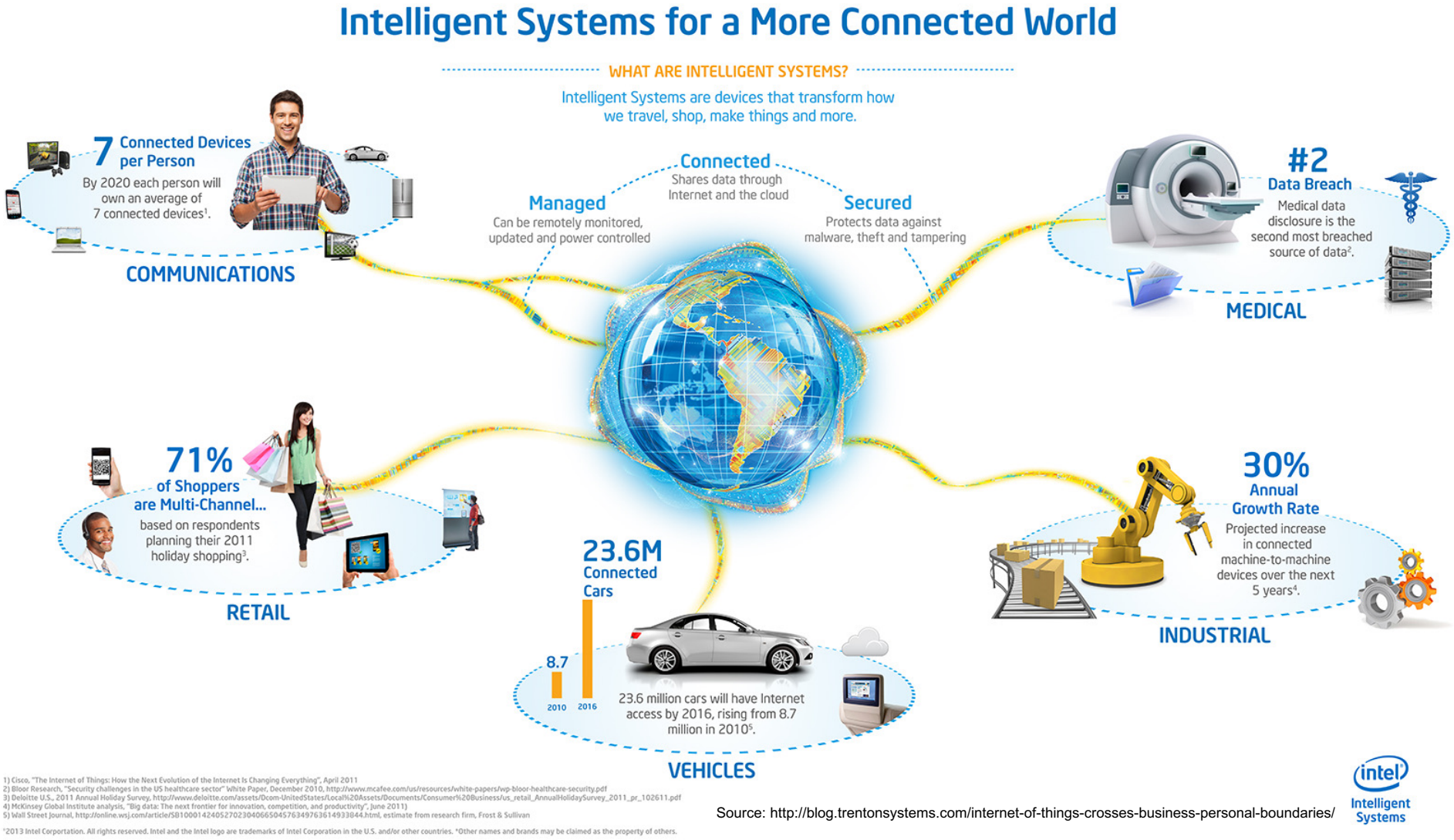
- The IETF work of the 6LoWPAN, ROLL and CoRE Working Groups (WGs) is of particular relevance for IoT/M2M.
- The CoRE WG is providing a framework for resource-oriented (i.e. RESTful) applications intended to run on constrained IP networks.
- The CoRE WG has defined a Constrained Application Protocol (CoAP) for the manipulation of Resources on a Device. The protocol supports the communications aspects of the M2M service layer.
- CoAP and related protocols will continue to be developed by the IETF.
- **CoAP is adopted by ETSI M2M**.
- The CoRE infrastructure includes a Link format (discovery mechanism), an event subscription mechanism and a scalable device proxy. Devices are viewed as RESTFul resources and can directly be interacted with using HTTP or CoAP protocol operations (PUT, GET, POST, DELETE, etc.).

Another relevant aspect: Interoperability

- Interoperability: the ability of equipment and systems from different vendors to operate together
- Smart objects must interoperate from the physical layer up to the application or integration layer
  - At physical level: agreement on physical frequencies, signal modulation, information rate
  - At network level: agreement on the format of the information that is sent and received over the physical channel; how nodes are addressed; how messages should be transported through a network of smart objects
  - At application or integration level: share a common view on how data should be entered or extracted from a smart object network, as well as how the smart objects should be reached from outside systems
- Imperative: technical architecture of smart objects defined to ease interoperability → IP-based architecture

Smart Things/Objects – Smart Environments

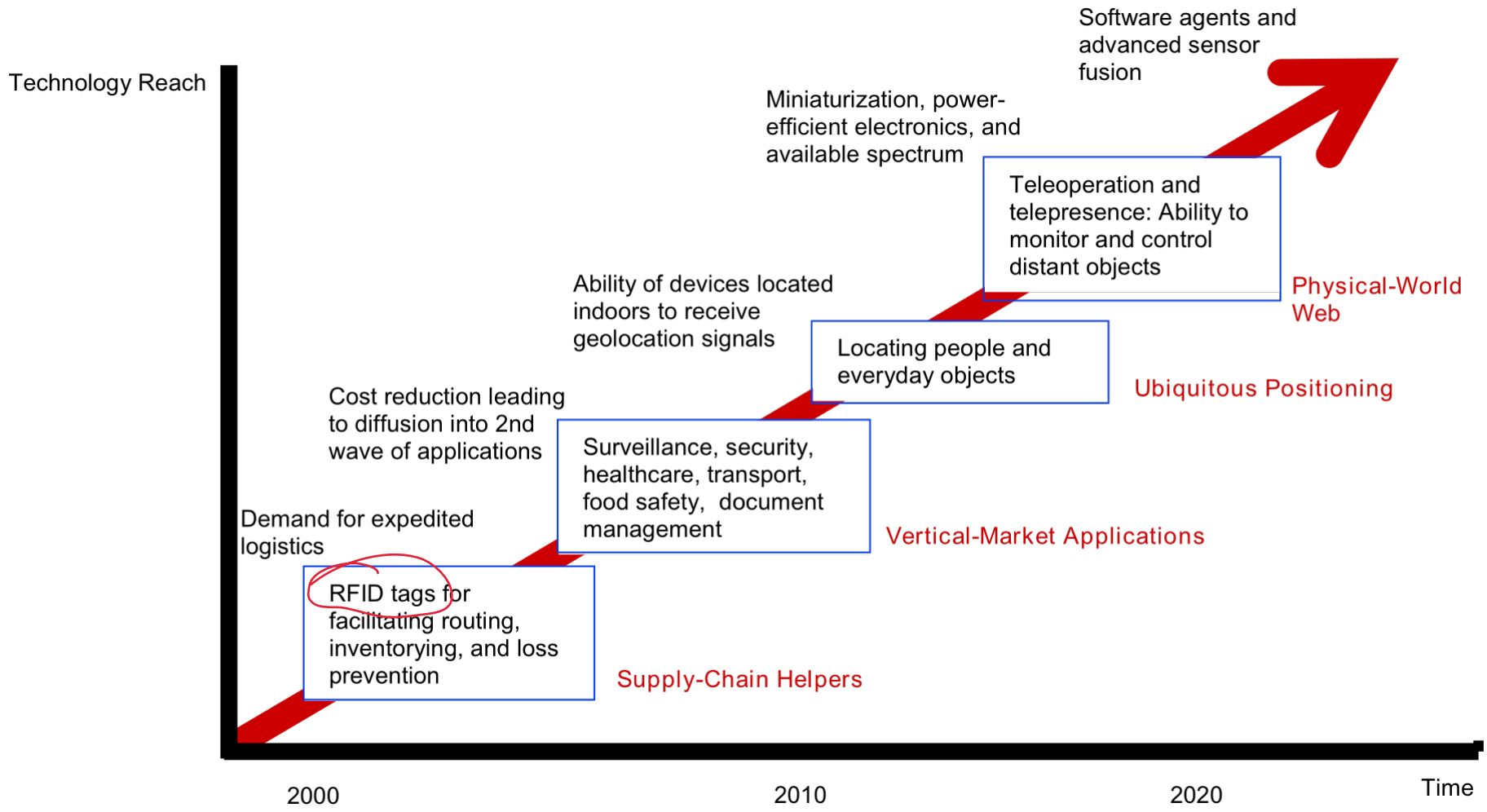
Opportunities



Source: http://blog.trentonsystems.com/internet-of-things-crosses-business-personal-boundaries/

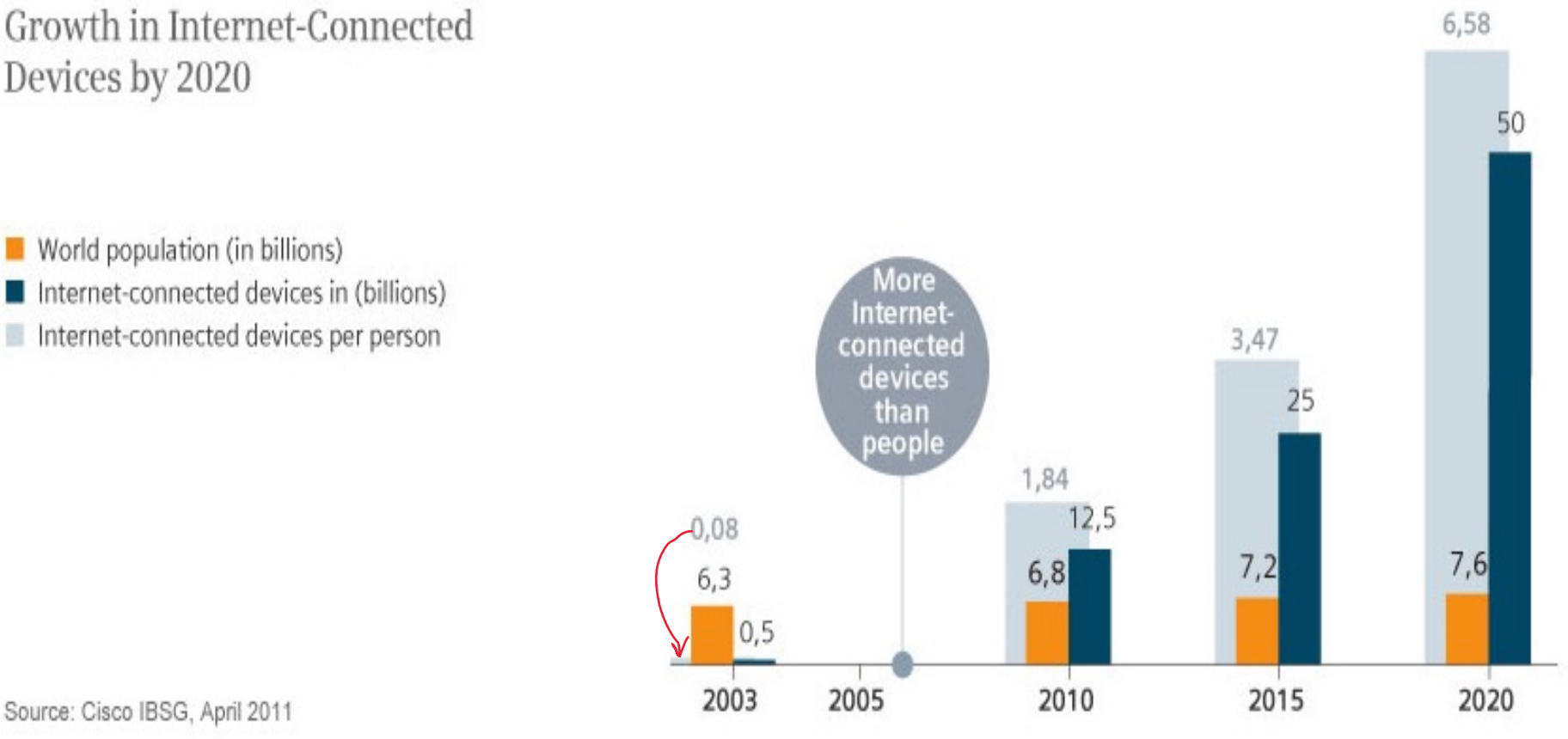
Technology trend

TECHNOLOGY ROADMAP: THE INTERNET OF THINGS



Source: SRI Consulting Business Intelligence

Internet-connected devices



Source: Siemens, http://www.siemens.com/innovation/apps/pof_microsite/_pof-fall-2012/_html_en/facts-and-forecasts-growth-market-of-the-future.html

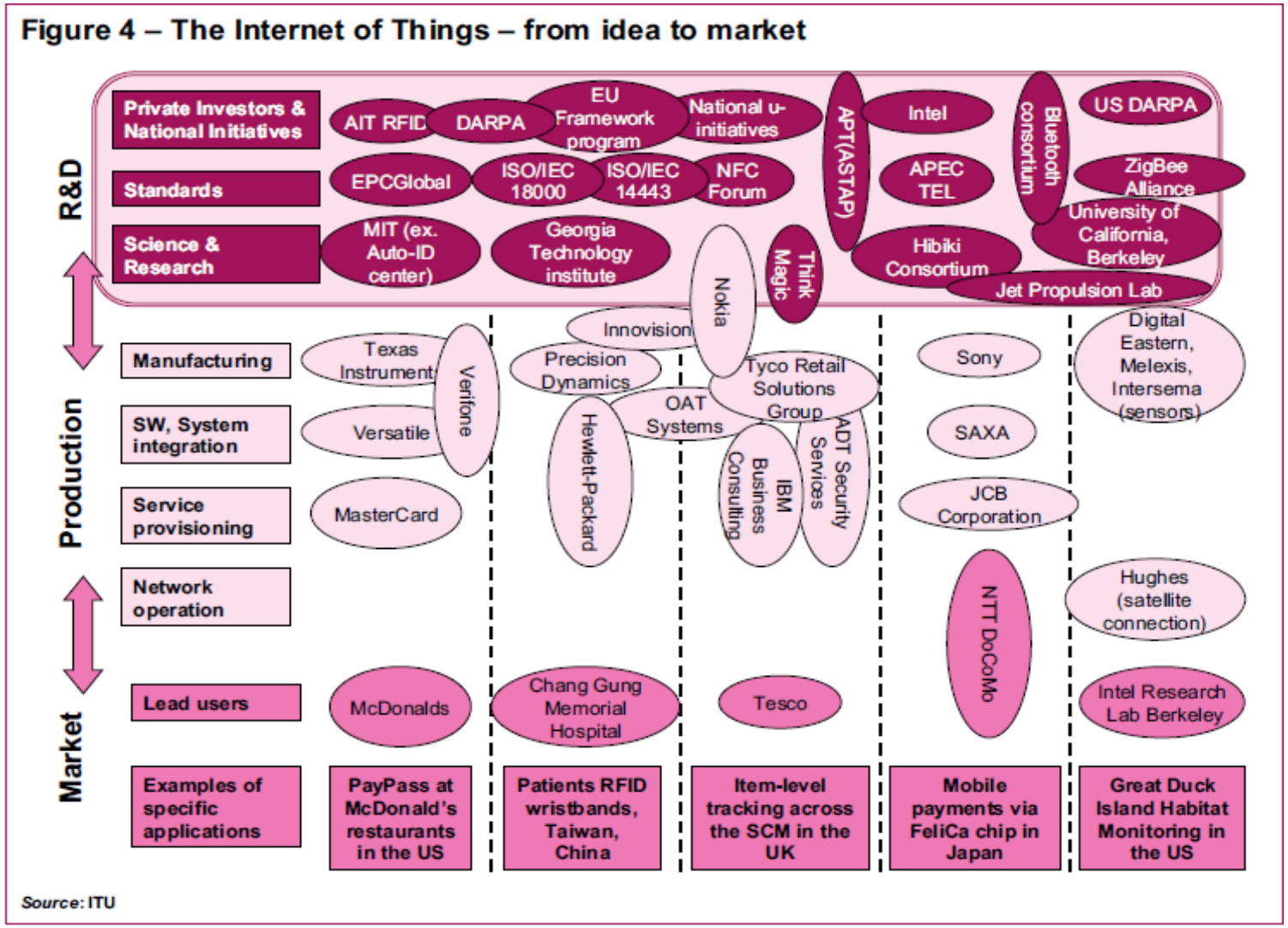
Global Data Generation

- Everyday around 20 quintillion (10^18) bytes of data are produced (Source: http://www-01.ibm.com/software/data/bigdata/).

- This data includes textual content (unstructured, semi-structured, structured) to multimedia content (images, video and audio), on a variety of platforms (enterprise, social media, and sensors).

- Big data!

State of the Art of IoT



An Outlook on Hardware

Main Hw Components

- The hardware of smart objects consists of four main components
  - [mouth/ears] Communication device (either a wireless transceiver with antenna or wired connection)
  - [brain] Microcontroller: defines the smart object's behavior
  - [hands] Set of sensors or actuators: interact with the physical world
  - [food] Power source: needed because the smart object contains electrical circuits

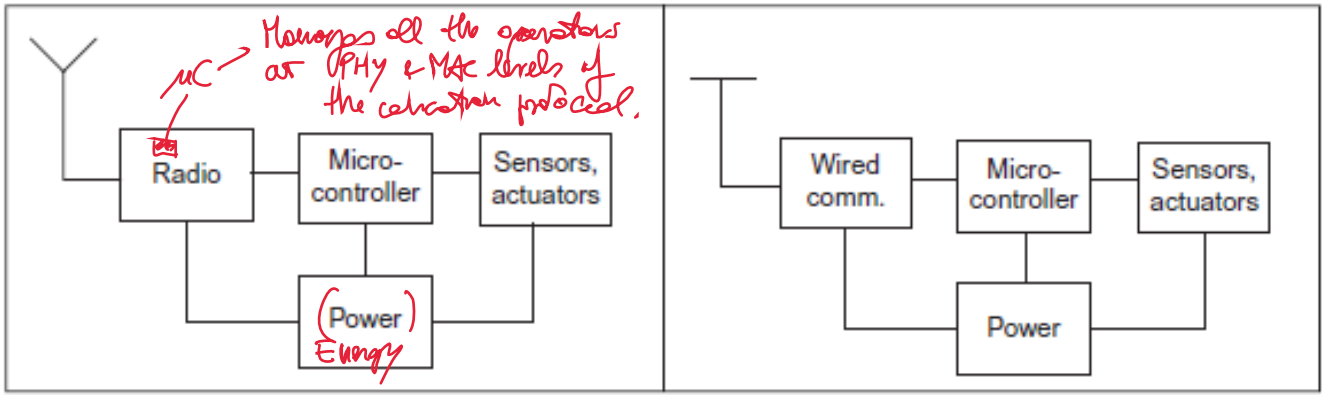


FIGURE 11.1

The hardware architecture of two smart objects: a radio-equipped, wireless smart object (left) and a smart object with wired communication (right).

An example: MicaZ

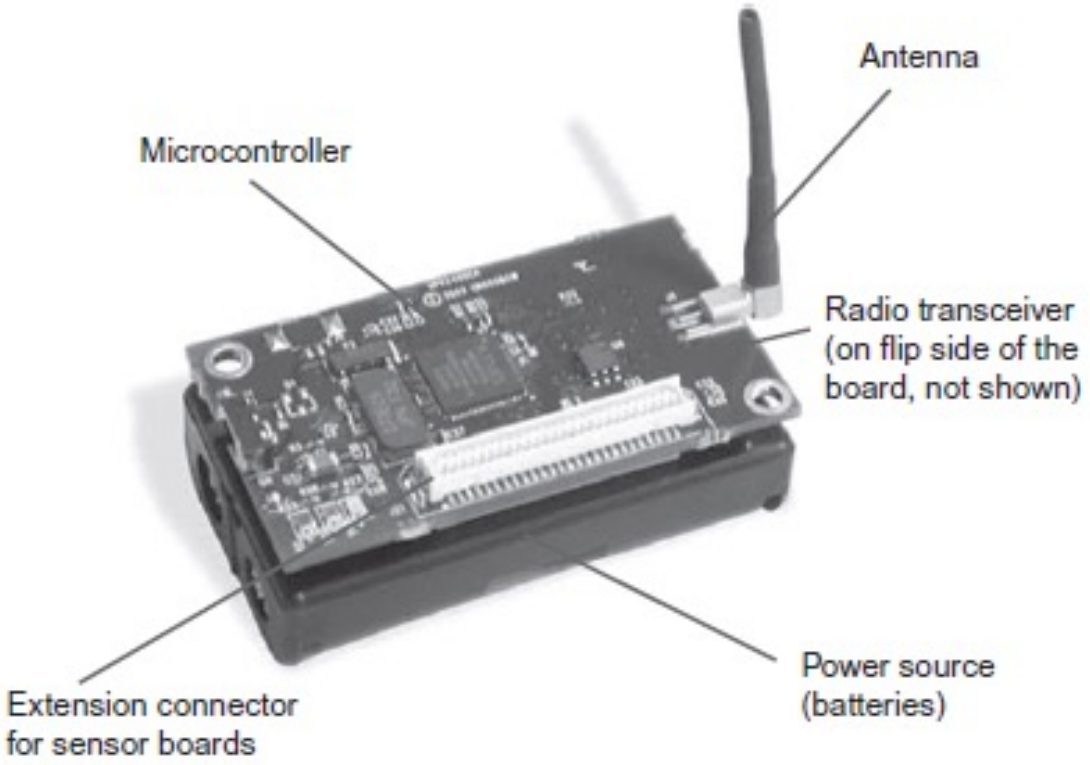


FIGURE 11.2

A MicaZ prototype board with a microcontroller, power source, communication device, and sensor connectors.

Photo courtesy of Crossbow Technology.

Communication Device

- Different types of radio transceivers have different amounts of built-in processing capabilities: (simple) send and receive individual bits of information into the air; (more powerful) package the information into packets, form headers, and even encrypt and decrypt the data using secure encryption methods
- Radio is usually the most power-consuming component (10 times the power consumption of the microcontroller or the sensors): modulation (tx) and demodulation (rx) –> idle listening as expensive as transmission
- Saving energy: duty cycling → need synchronization (see later)

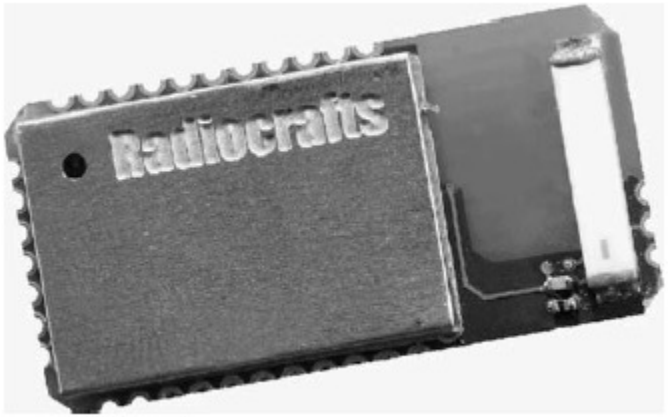


FIGURE 11.3
Texas Instruments CC2430 single-chip radio transceiver with integrated 8051 microcontroller and on-board antenna manufactured by Radiocrafts. The size of the board is $1.2 \times 1.0\,cm^2$.
Photos courtesy of Radiocrafts.

Example of system-on-chip (SoC)

Microcontroller

- Runs the software of the smart object & connects radio with sensors/actuators
- Microcontroller: microprocessor with built-in memory, timers, and hw for external connections
- Microcontrollers used in smart objects (cost lower than 10$) much smaller than those used in PCs
  - Smart object: a few Kbytes of on-chip memory ; clock rate of a few MHz
  - PC: Gbytes of memory; clock rate of several GHz
- Memory: ROM (behaviour) and RAM (temporary sw for tasks)

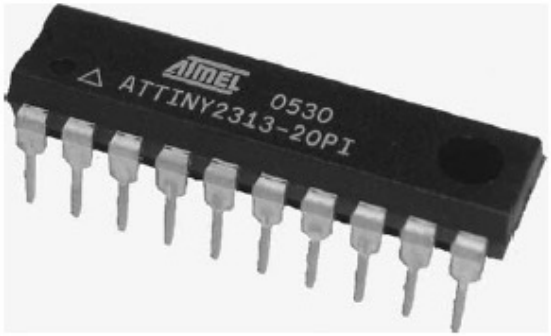


FIGURE 11.4

An Atmel ATTINY 2313 smart object microcontroller with 20 pins. The ATTINY 2313 has 2 kB of ROM and 128 bytes of RAM. This represents the low end of smart object microcontrollers.

Table 11.1 Microcontrollers Used in Smart Objects

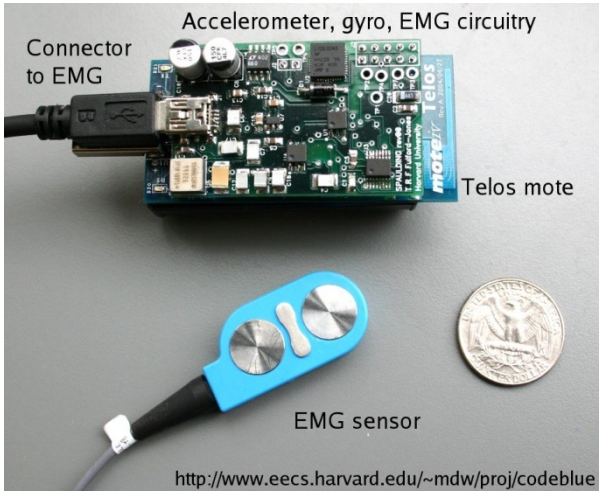
Name	Manufacturer	RAM (kB)	ROM (kB)	Current consumption (active/sleep), mA
MSP430xF168	Texas Instruments	10	48	2/0.001
AVR ATmega128	Atmel	8	128	8/0.02
8051	Intel	0.5	32	30/0.005
PIC18	Microchip	4	128	2.2/0.001

Note: Each manufacturer has several models of each device. This table lists only one example from each manufacturer.

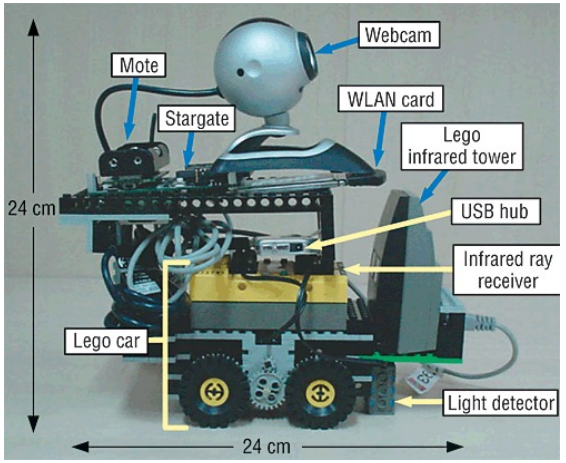
Sensors and Actuators

- Sensors: sense the environment
- Actuators: affect or change the environment
- From very simple (temperature sensor) to complex (ultrasonic range device or camera for fence monitoring)

Video recording



ElectroMioGraphy



Energy Power Sources

- Most common power source: battery (Lithium, non rechargeable)
  - (+) lifetime of many years on a single standard AA-size battery (low-power electronics and power-saving software)
  - (-) difficult to recycle
- Power scavenging: harvest from the environment (solar cells, piezoelectricity, electromagnetic energy-RFID)

FIGURE 11.5
PowerCast P2100 module converts radio energy from the air into electricity that powers an electrical circuit. The module requires an external antenna to pick up the radio signals (not shown).

Table 11.2 Different Power Sources for Smart Objects, Their Maximum Current Draw, and the Amount of Charge They Store

Power source	Typical maximum current (mA)	Typical charge (mAh)
CR2032 button cell	20	200
AA alkaline battery	20	3000
Solar cell	40	Limitless
RF power	25	Limitless

Energy Consumption

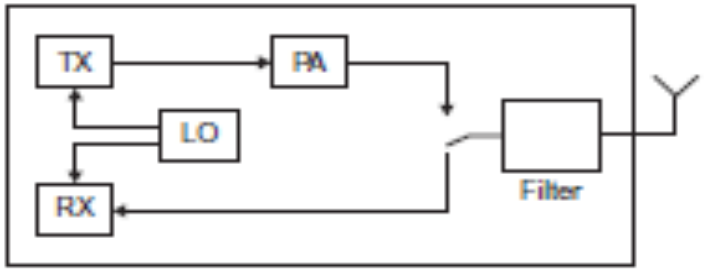


FIGURE 11.17

Schematic drawing of a radio transceiver.

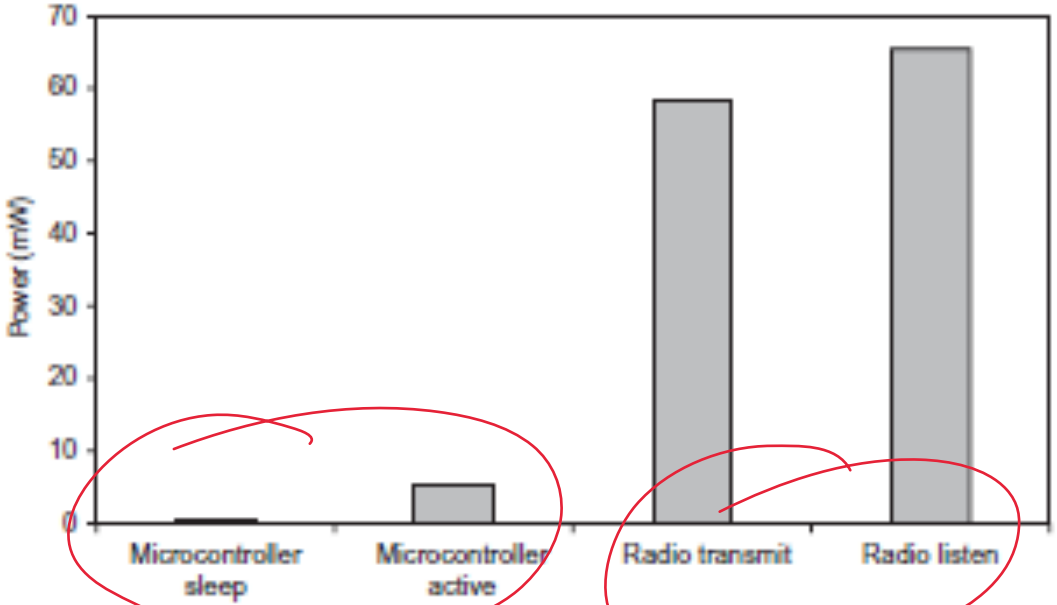


FIGURE 11.16

Power consumption of the microcontroller and the radio on the Tmote Sky smart object prototyping board. The power consumption of the microcontroller sleep mode is so low that it is not visible in the graph.

An Outlook on Software

Programming a Smart Object

- Programs written in a high-level programming language (e.g., C, maybe Java) and compiled with a compiler to machine code for the microcontroller
- Machine code written into ROM, run when the smart object is powered on
- Operating systems are used by smart objects (see next slide)
- Memory footprints makes programming (e.g., communication protocol stack) challenging (see later)

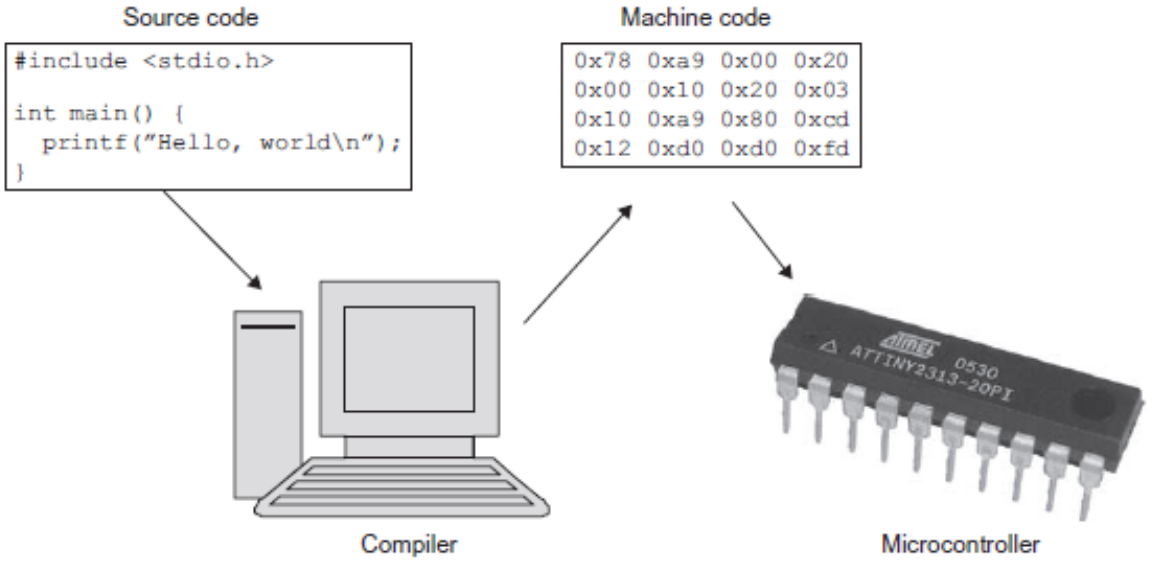


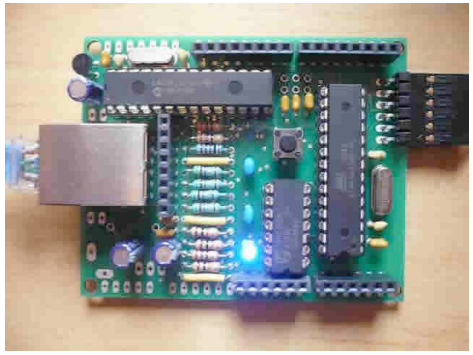
FIGURE 11.8

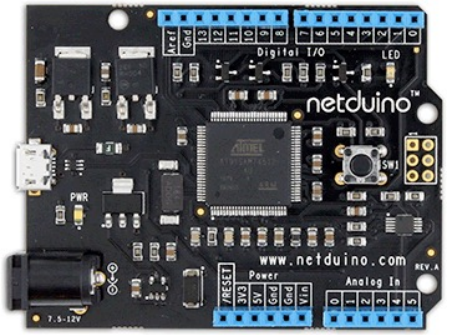
The process of software development for a smart object. Source code is compiled to machine code that is written to the ROM of the smart object microcontroller.

Operating Systems

- Very different from general-purpose operating systems
  - Tailored to specific requirements of smart objects and hardware constraints (memory constraints and processing speed)
  - No graphical interface (hidden deeply within the microcontroller)
- Main operating systems (all open-source)
  - **Contiki** (2003-, SICS-Thingsquare): first OS which guarantees IP communication with uIP TCP/IP stack (5 kB), incorporating also uIPv6 (11 kB) in 2008; useful simulator (Cooja); many components used in industry (uIP TCP/IP, protothread programming abstraction); written in C (ported to more than 12 microprocessors/controllers); direct communication with other IP-based application and web services (e.g., Twitter, mail)
  - **TinyOS** (2000-, Berkeley-Stanford): focus on wireless sensor networks and academia; used uIP for IP communication and has recently incorporated independent IPv6 implementation; written in nesC; event-driven programs
  - **FreeRTOS**: designed for embedded systems, providing real-time guarantees (control applications); provides TCP/IP support (uIP and lwIP); uses a preemptive, multi-threaded (multiple programs simultaneously running-large memory) programming model; ported to over 50 microprocessors/controllers

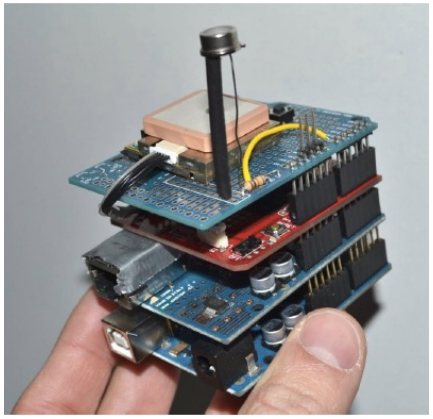
IoT Devices ("homegrown")





Netduino

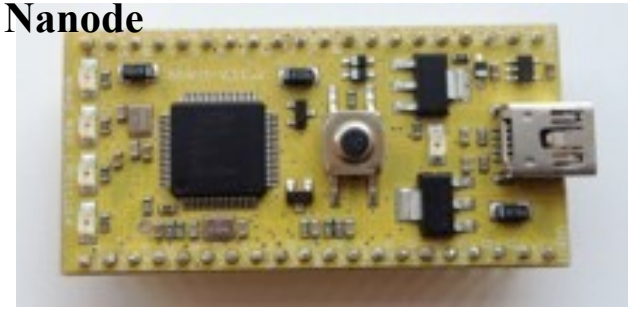
COTS
Comercial
off
the
Shelf



52North SenseBox light

Arduino + Ethernet + GPS ? ~ 100 €

Nanode



mbed.org ARM® Cortex™-M0

- Open source electronic platforms
- Must add sensors, comms, positioning
- Prices dropping rapidly

Twine

WiFi, temperature and vibration sensors, $100+ plus cloud service

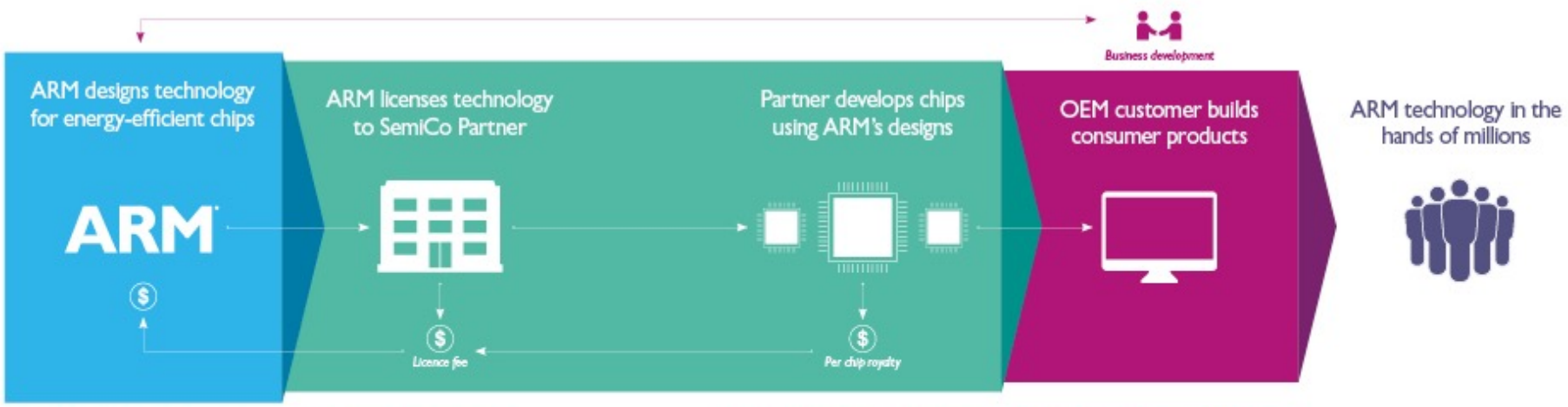
© 2012 Open Geospatial Consortium

Illustrative Spectrum of Embedded Resources: Cortex

MCU capability	Class	System Cost $ (IC + required crystals, memory, etc)	RAM (B)	Flash (B)	OS	Connectivity	Application	Capability
High	Low A	12	>16M	>64M	Linux, Android	Full IP stack	Gateway, local hub, smart pen	Full internet stack, rich OS (Linux, Android), typical gateway device
Med High	M4	8	1M	1M				
Med	M3, M4	5	48k	256k	RTOS, Embedded OS	6LowPAN	Smart watch, washing machine controller	COAP + DTLS, application developed using rich ecosystem of frameworks and libraries, capable of some audio and GUI
Low	M0, low M3	2	10k	64k	Embedded OS, none	6LowPAN	Light fitting, motion sensor	COAP (+DTLS?), 6LowPAN (just), quite constrained application
Ultra low	M0+	<1	<4k		none	Proprietary low-rate / short range	Single use temperature sensor, mm^3	Highly constrained, carefully optimised code, uplink via an application gateway device

The Architecture for the Digital World® ARM®

ARM Business Model



1. ARM designs technology to go into energy-efficient chips for a broad range of end markets. ARM licenses each design to multiple semiconductor companies. Every company pays an upfront licence fee to gain access to the design.

2. The semiconductor company will incorporate the ARM technology design into their chip. It can take 2–3 years to build a chip and a further year for an OEM to build their product, such as a digital TV or mobile phone, containing the chip. When the chip starts to ship, ARM receives a royalty on every chip that uses the design. Typically our royalty is based on the price of the chip.

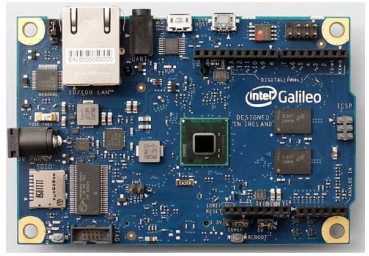
3. Each ARM design is suitable for a wide range of end applications and so can be re-used in different chip families addressing multiple markets. Each new chip family generates a new stream of royalties. An ARM design may be used in many different chips and may ship for over 20 years.

4. ARM's flexible business model has proven to be suitable for different technologies, end products, licensing strategies and a broad range of ecosystem Partners.

Deployment of an IoT testbed

150+ nodes

(3/4G, WiFi, 802.15.4, SubGHz, UWB, BLE,NFC,...)



intel Galileo

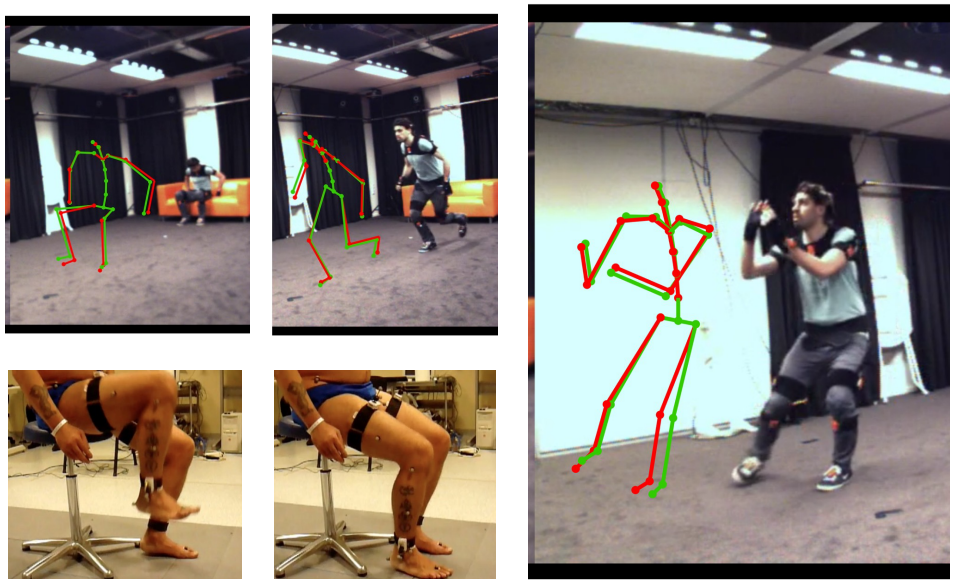
Affiliated with the Intel Galileo University Program

Body Sensor Networks

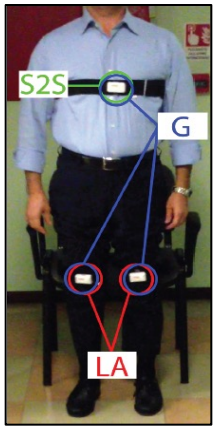
A Body Sensor Network (BSN) is a sensor network composed by nodes placed on a user body.The network nodes are, typically, devices supplied with inertial (i.e., accelerometers and gyroscopes) and magnetic (i.e., magnetometers) sensors.

Applications:

- motion capture
- posture recognition
- activity classification
- tele-rehabilitation (Parkinson's Disease)
- rehabilitation monitoring (post-stroke)



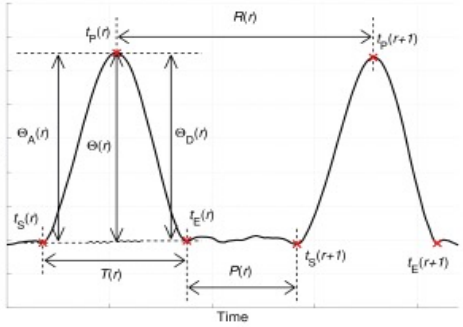
BSN-based Evaluation of PD Motor Tasks

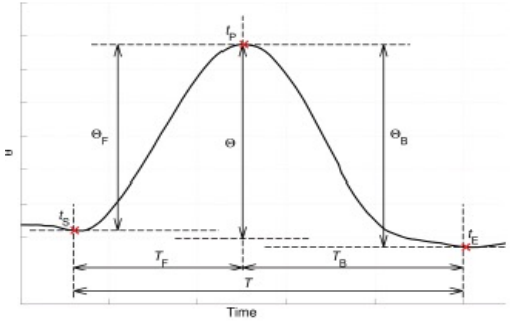


- **Main goal**: provide an automatic assessment of PD motor tasks
  - 3 UPDRS tasks: Leg Agility (LA), Sit-to-Stand (S2S), and Gait (G)
  - More objective and continuous evaluation of the symptoms
- **Hardware:** low-complexity BSN (3 IMUs - Shimmer)
- **Kinematic Characterization:** extraction of task-relevant features in both time and frequency domains

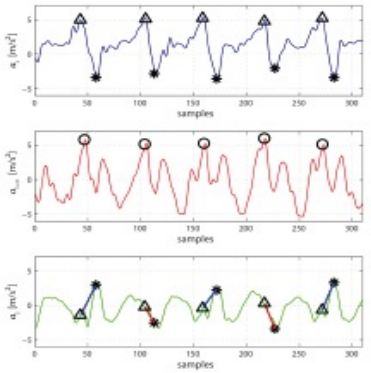
ISTITUTO AUXOLOGICO ITALIANO
Istituto di ricovero e cura a carattere scientifico

LA

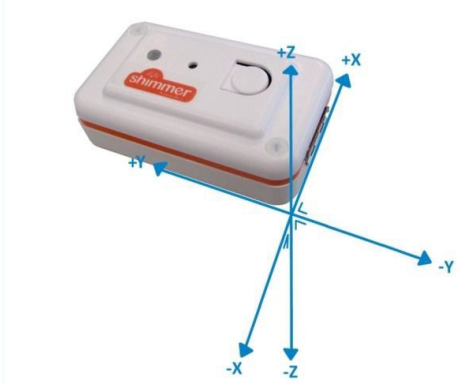




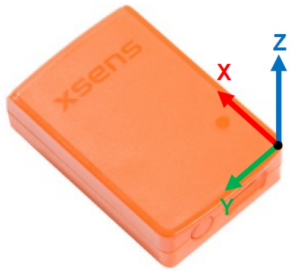
S2S



IMUs @ IoT Lab



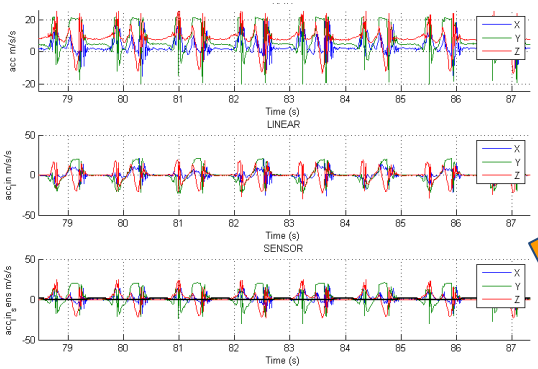
- Shimmer 2r node
- Small-sized, low-power, Bluetooth interface
- 9 DOF: 3-axial accelerometer, 3-axial gyroscope, 3-axial magnetometer
- Sampling Rate: 102.4 Hz

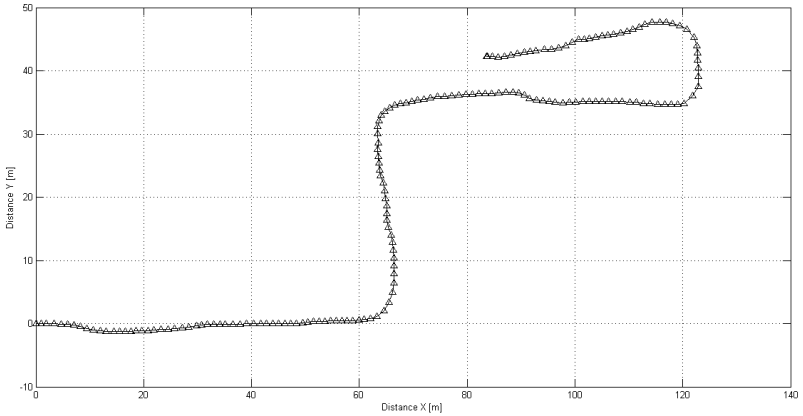


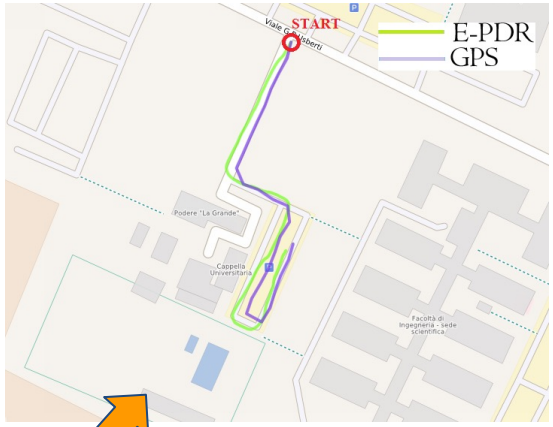
- 6 MTw Awinda IMU (accelerometer, gyroscope, magnetometer, barometer)
- Awinda Full Body straps set
- Awinda USB receiving dongle
- 6 slot Xsens docking station

- LPMS-B2
- Size: 39 x 39 x 8 mm
- Bluetooth Low Enery (LE) 4.1
- Communication distance < 20 m
- Orientation range Roll: ±180°; Pitch: ±90°; Yaw: ±180°
- Resolution < 0.01°
- Accuracy < 0. 5°(static), < 2° RMS (dynamic)
- Accelerometer 3 axes, ±2 / ±4 / ±8 / ±16 g, 16 bits
- Gyroscope 3 axes, ±125 / ±245 / ±500 / ±1000 / ±2000 °/s, 16 bits
- Magnetometer 3 axes, ± 4 / ± 8 / ± 12 / ± 16 gauss, 16 bits

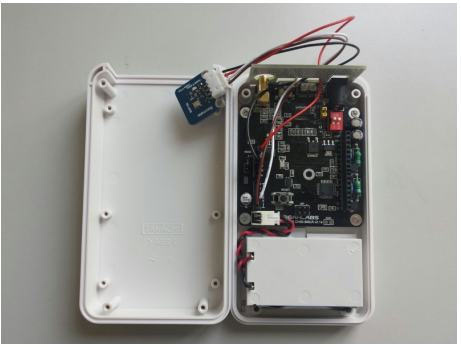
Inertial navigation and tracking: outdoor

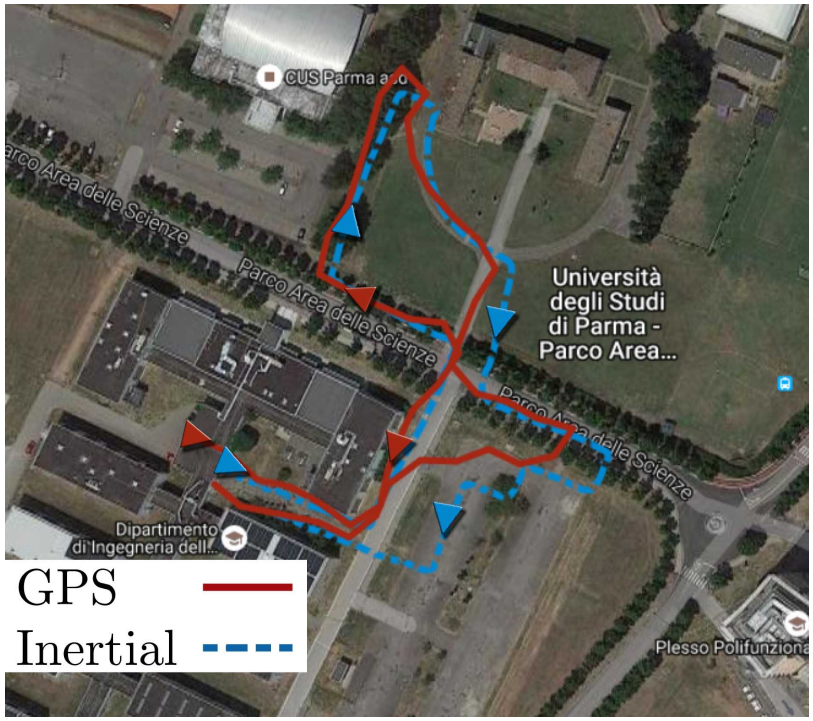






Evolutions of Inertial Navigation





- Hybrid inertial/barometric sensor → multi-floor navigation
- Navigation and tracking on non-flat surfaces (e.g., cross training)

A Demo (IPIN 2016, October 2016)

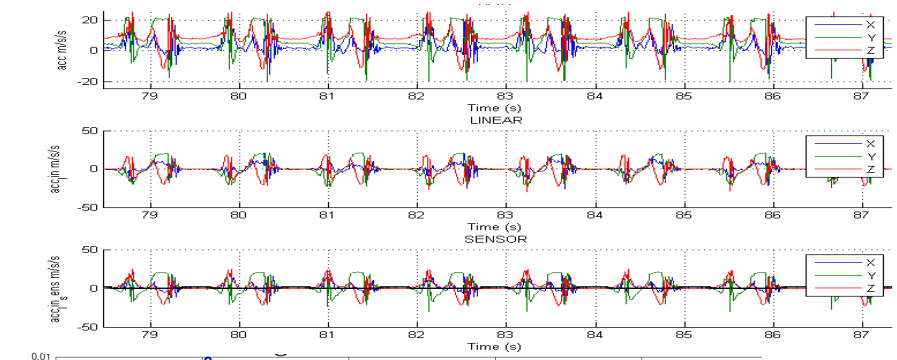
IoT Lab @ IPIN 2016
SEVENTH INTERNATIONAL CONFERENCE ON
INDOOR POSITIONING AND INDOOR NAVIGATIO
October 4-7, 2016 / Alcalá de Henares, Madrid, Spain

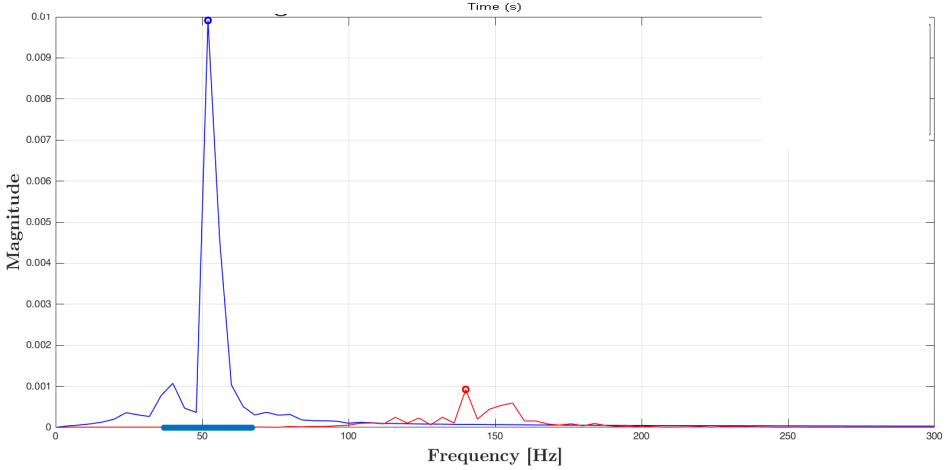
Track 2 Competition
3-rd Place Classified

- IPIN 2016 indoor localization competition (Alcalá de Henares, Madrid)

- IPIN tracks
  - Track 1: Smartphone-based
  - Track 2: Pedestrian dead reckoning positioning
  - Track 3: Smartphone-based (off-site)
  - Track 4: Indoor mobile robot positioning

Vibration Analysis



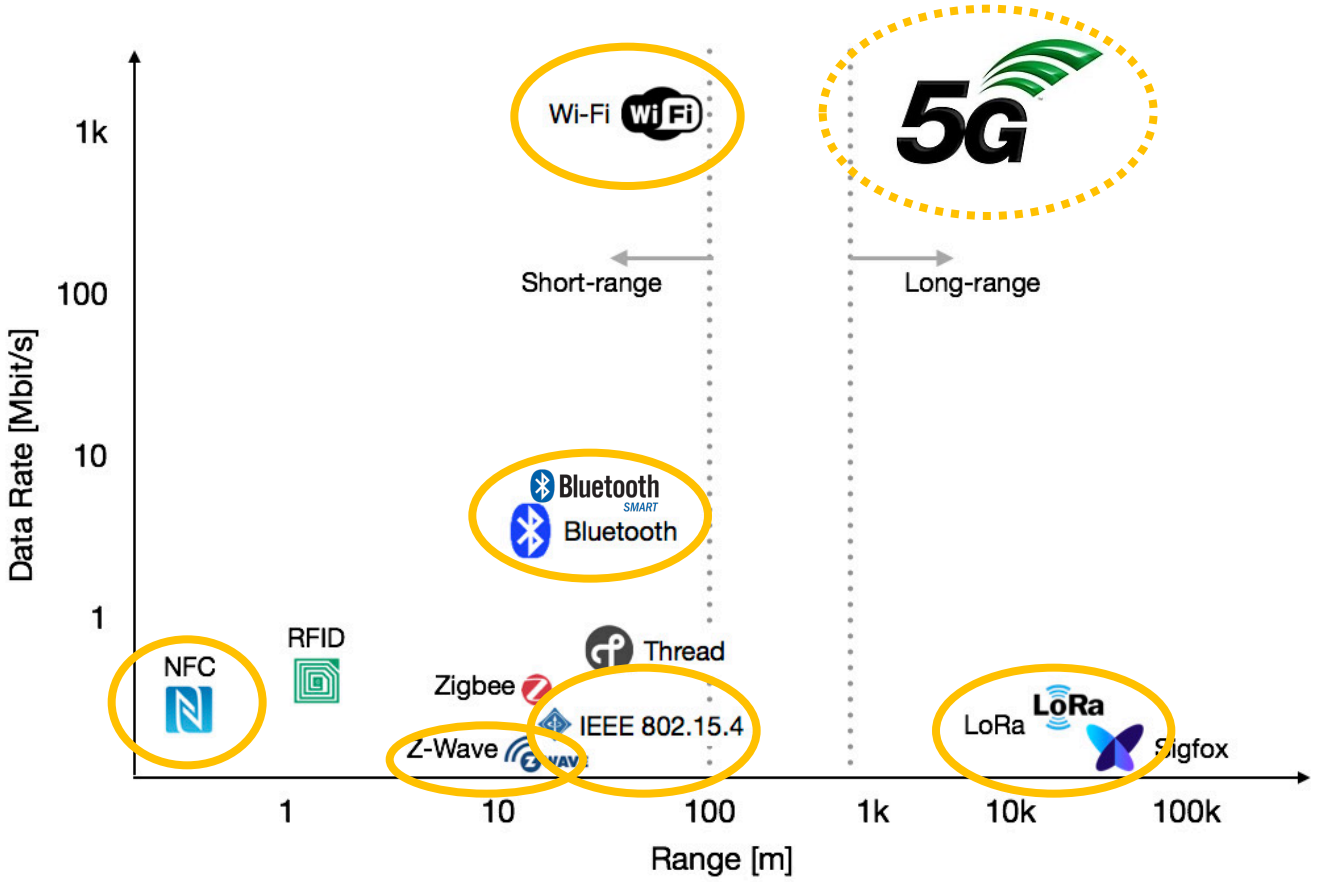


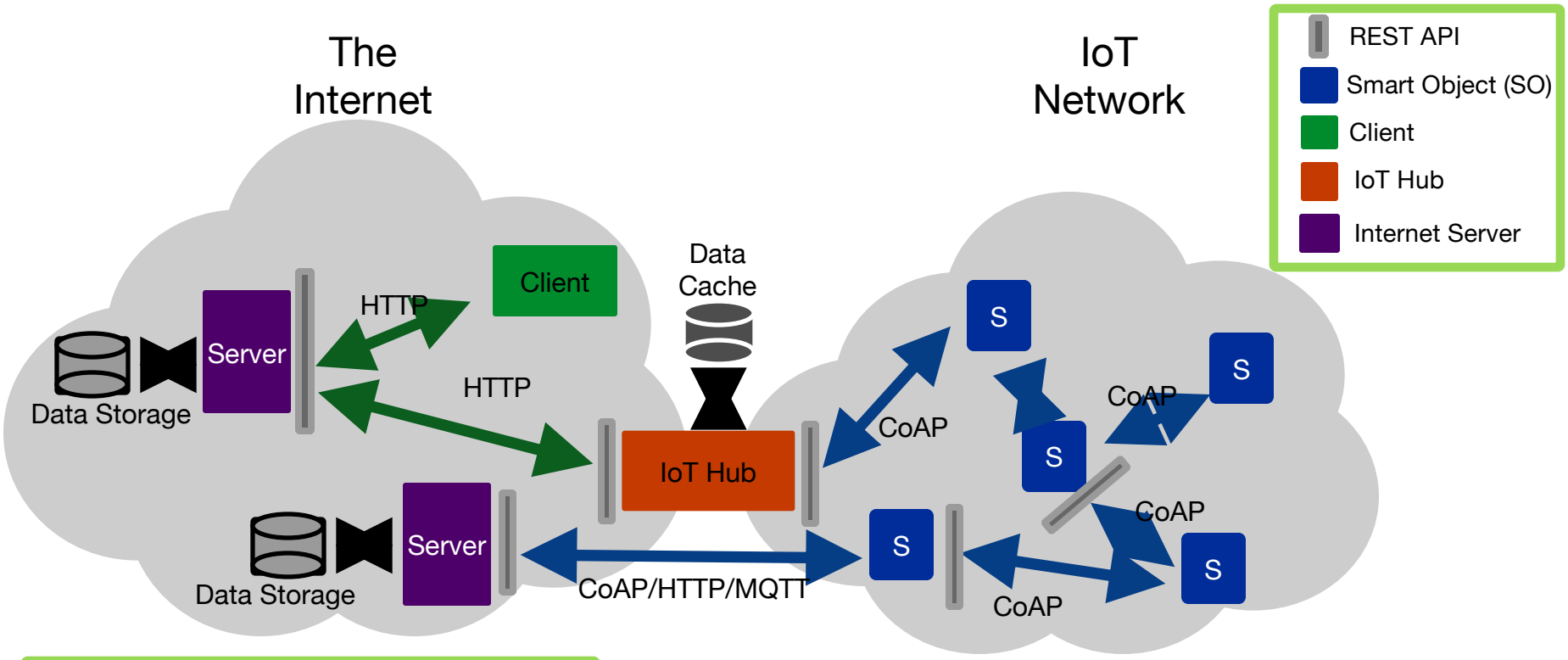
- Simple and non-invasive
- Combine an inertial sensor with an embedded system
- Extension: combine with audio sensor

Application: real-time RPM estimation

- Hybrid time/frequency processing

“Our” Wireless Communication Technologies

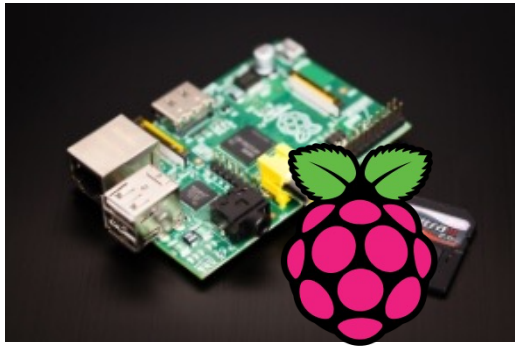




Data Storage, Processing, Aggregation and Distribution

Data Source/Stream, manage Incoming and Outgoing requests/responses SO

Infrastructure nodes



RaspberryPi

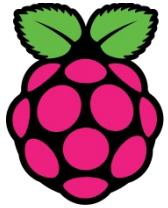
- 802.11
- Ethernet
- Camera

Arduino Yùn

- 802.11
- Ethernet

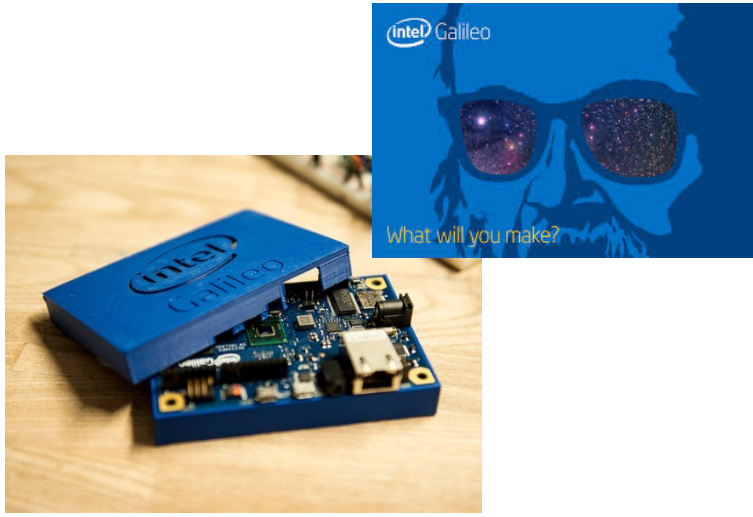
12 AP Cisco

- 802.11
- Ethernet



Gateway = RaspeberryPi & Telos B

- 802.15.4
- 802.11
- Ethernet



INTEL Galileo

- Ethernet
- WiFi (dongle)

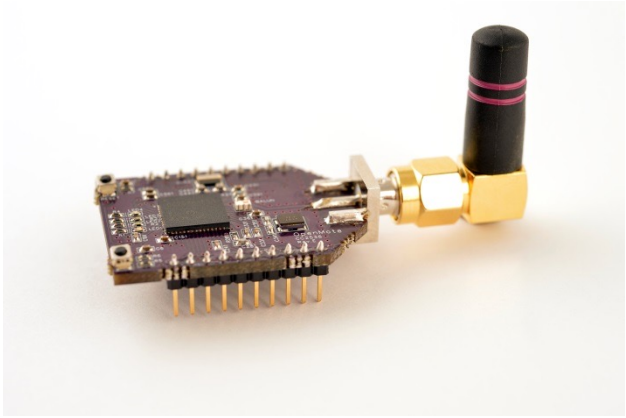
Mobile nodes

Telos B

- 802.15.4
- Contiki

OpenMote

- 802.15.4
- Contiki



UDC. 2

Zolertia Z1

- 802.15.4
- Contiki

UNIVERSITÀ DI PARMA

IoT Lab

→ Lec. 3

Transport Layer Protocols: UDP and TCP

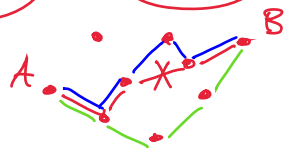
Main sources

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapters 2 and 6.

- Princeton (COS 461: Computer Networks, Spring 2006) and Virginia Tech (Internet Teaching Lab, http://www.cs.virginia.edu/~itlab/) lectures

Original TCP/IP Goals (70's)

- Internet communication must continue despite loss of networks or gateways (" in the presence of link or node failures " in today's terms).



- The Internet must support multiple types of communication services.
- Internet architecture must:
  - accommodate a variety of networks (" networks " means link and physical layers)
  - permit distributed management of its resources
  - be cost-effective
  - permit host attachment with little effort
- Resources used in the Internet architecture must be accountable .

Scalability of TCP/IP

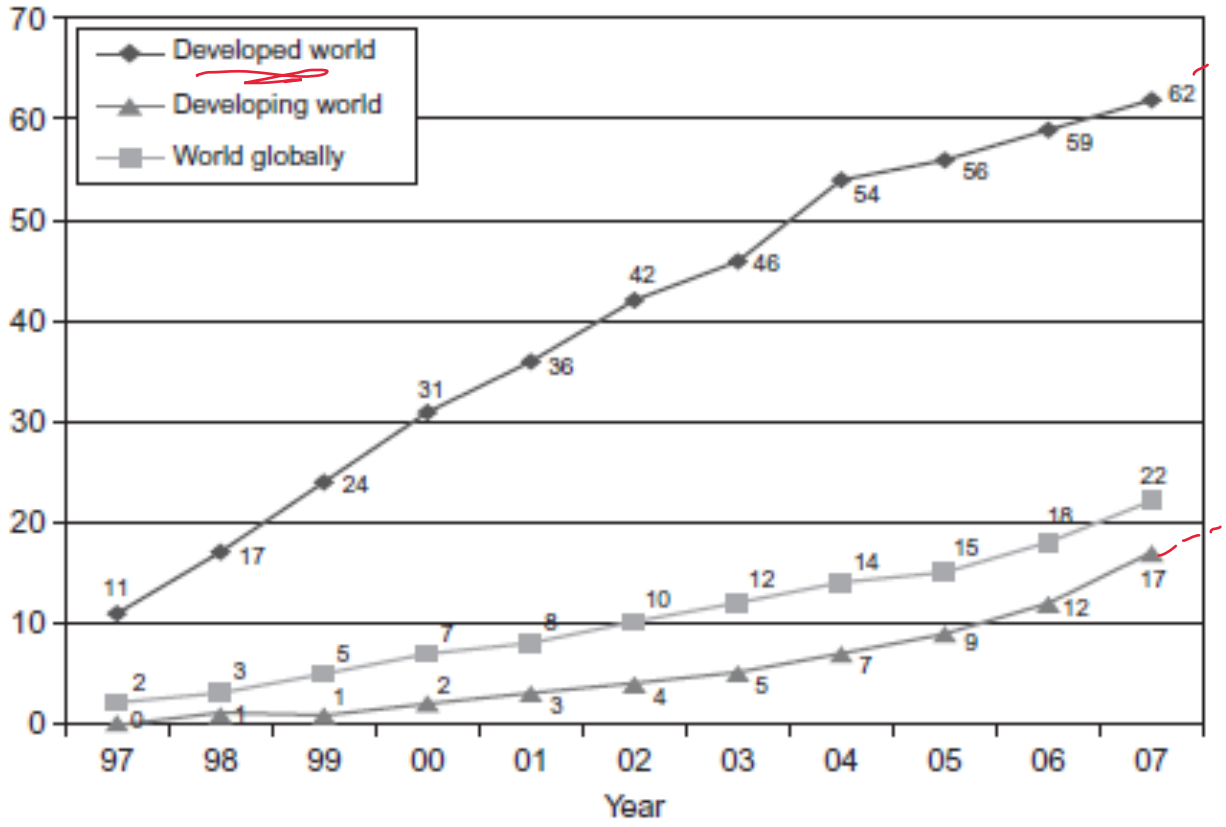


FIGURE 2.1

Level of penetration of the Internet regarding user number.

Protocol Stack (layers)

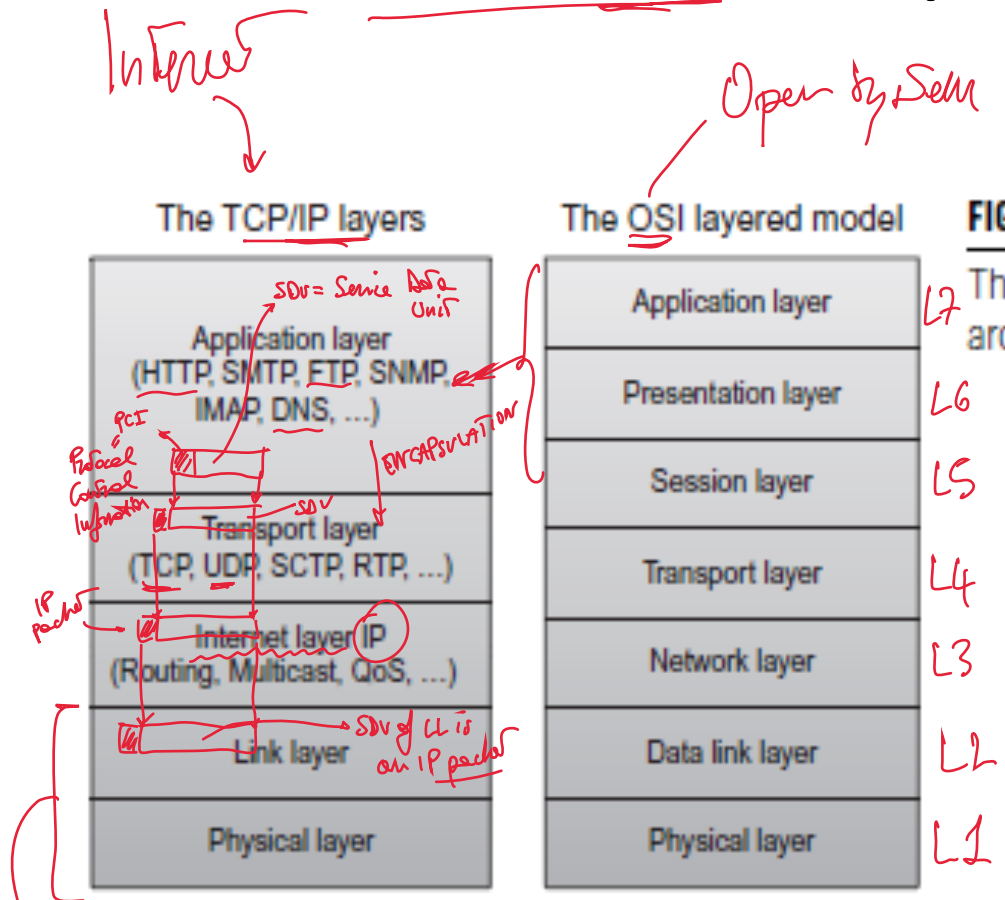
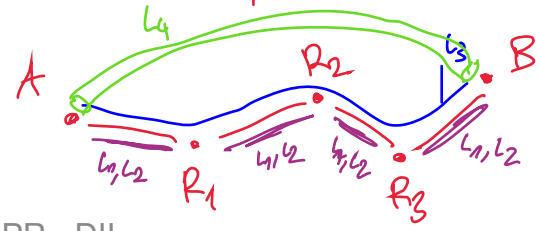


FIGURE 2.2
The concept of layering in the TCP/IP and OSI architectures.



Link Layer

- Usually refers to the physical and data link layers (the use of the PHY/MAC acronym is fairly common).
- Responsible for forwarding the IP packet on a link between two devices. This involves several functions such as: medium access control (MAC), error detection and (sometimes) retransmission, and flow control.
- Link layer protocol information is added in the form of a frame that carries the IP packet.
- Some links provide a very limited set of functions whereas others implement fairly sophisticated services that often include a link layer " routing " function.

Internet/Network Layer (IP)

- Responsible for providing an unreliable service for sending a packet between a source and a destination across the network, where host and routers are uniquely identified by their IP (IPv4 or IPv6) addresses, using a hierarchical addressing scheme.

- Routing is one of the main tasks accomplished by the IP layers

- Protocols such as ICMP (Internet Control Message Protocol) and IGMP (Internet Group Management Protocol) for multicast traffic are both considered part of the IP layer.

Transport Layer

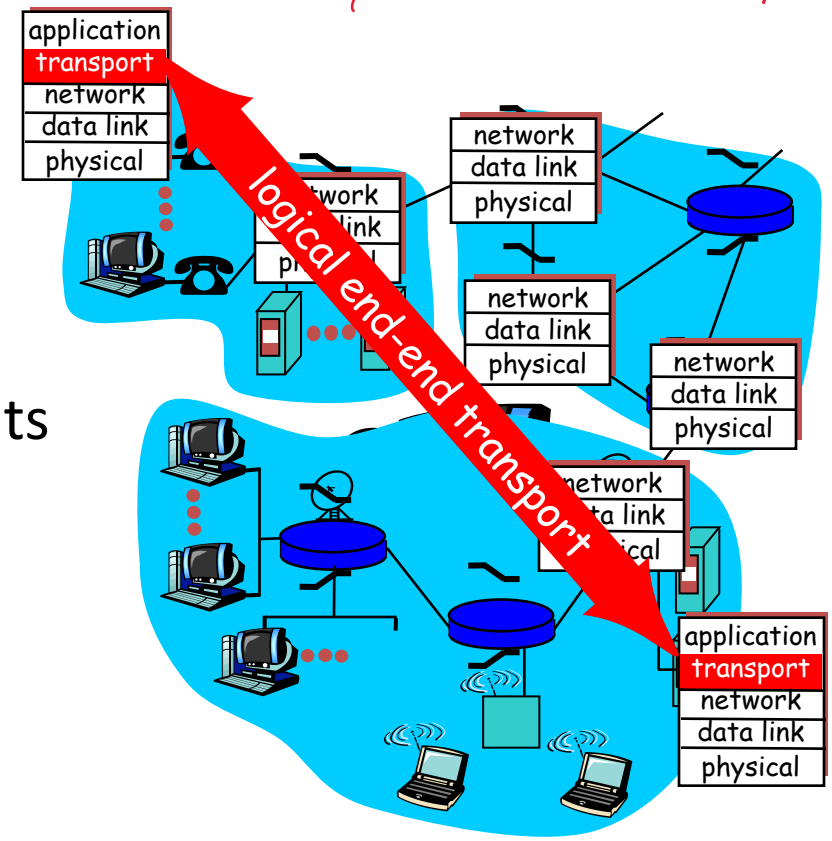
- Responsible for end-to-end communication between two devices.
- TCP (states are maintained at the end devices) provides a reliable transport mechanism with error detection and retransmission, flow control using dynamic windowing techniques, security mechanisms, and so forth.
- UDP (stateless) is mostly used for application addressing and optional error detection (done by an optional checksum in IPv4; note that the UDP checksum is mandatory in IPv6).
- Other transport protocols:
  - the message stream transport protocol SCTP (Stream Control Transmission Protocol) offers additional capabilities such as the bundling of multiple user messages in a single SCTP packet, the support of multi-homing, and so forth.
  - Real-time Transport Protocol (RTP) is another transport protocol designed for realtime applications such as streaming audio traffi c and video .

Application Layer

- Refers to higher level protocol(s) that supports the applications.
- The list of application layer protocols is fairly long, but few well known include:
  - File Transfer Protocol (FTP)
  - Trivial File Transfer Protocol (TFTP)
  - Simple Network Management Protocol (SNMP)
  - HypertextTransport Protocol (HTTP)
  - Network News Transfer Protocol (NNTP)
  - Telnet

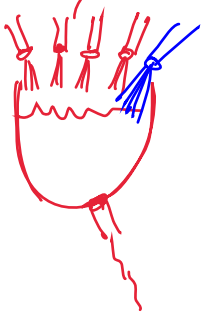
Transport Protocols

- Provide *logical communication* between application processes running on different hosts
- Run on end hosts
  - Sender: breaks application messages into segments, and passes to network layer
  - Receiver: reassembles segments into messages, passes to application layer
- Multiple transport protocol available to applications
  - Internet: TCP and UDP



Internet Transport Protocols

- Datagram messaging service (UDP)
  - No-frills extension of "best-effort" IP
- Reliable, in-order delivery (TCP)
  - Connection set-up
  - Discarding of corrupted packets
  - Retransmission of lost packets
  - Flow control
  - Congestion control
- Other services not available
  - Delay guarantees
  - Bandwidth guarantees



Transport Protocols in the Internet

- The Internet supports 2 transport protocols

UDP - User Datagram Protocol

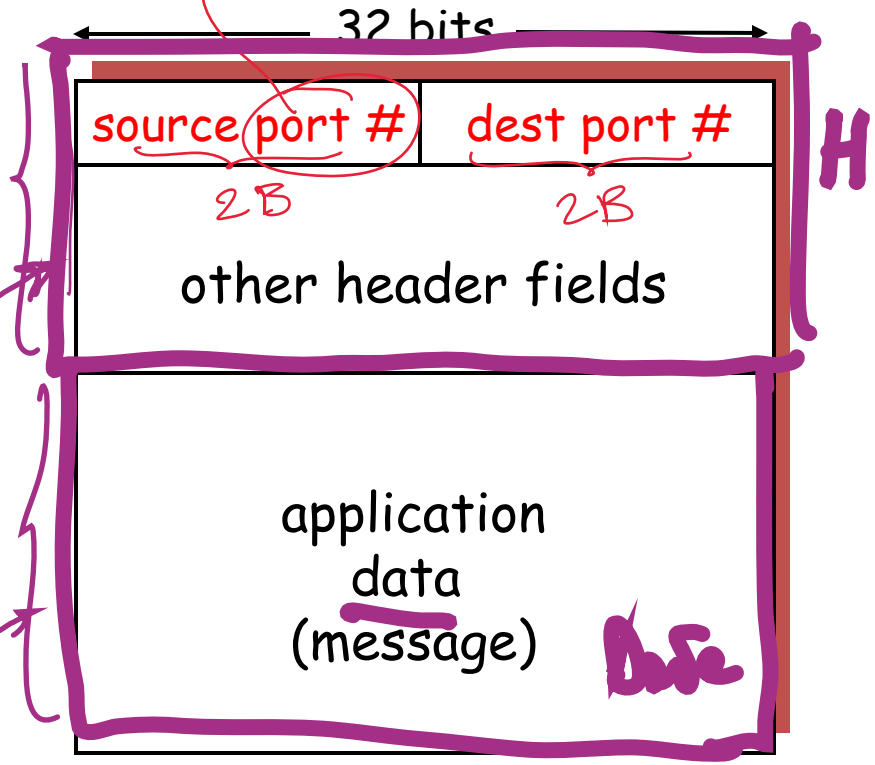
- datagram oriented
- unreliable, connectionless
- simple
- unicast and multicast
- useful only for few applications, e.g., multimedia applications
- used a lot for services
  - network management (SNMP, Simple Network Management Protocol), routing (Routing Information Protocol, RIP), naming (DNS), etc.

TCP - Transmission Control Protocol

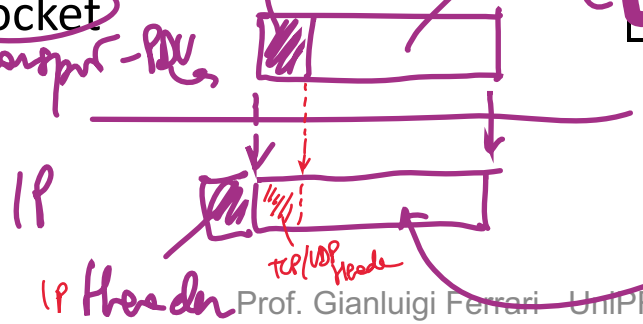
- stream oriented
- reliable, connection-oriented
- complex
- only unicast
- used for most Internet applications:
  - web (http), email (smtp), file transfer (ftp), terminal (telnet), etc.

Multiplexing and Demultiplexing

- Host receives IP datagrams
  - Each datagram has source and destination IP addresses
  - Each datagram carries one transport-layer segment
  - Each segment has source and destination port number
- Host uses IP addresses and port numbers to direct the segment to appropriate socket



TCP/UDP segment format



User Datagram Protocol (UDP)

- UDP is the simplest protocol in the TCP/IP suite
- Lightweight communication between processes
  - Avoid overhead and delays of ordered, reliable delivery
  - Send messages to and receive them from a socket
- UDP in a nutshell:
  - IP plus port numbers to support (de)multiplexing
  - Optional error checking on the packet contents

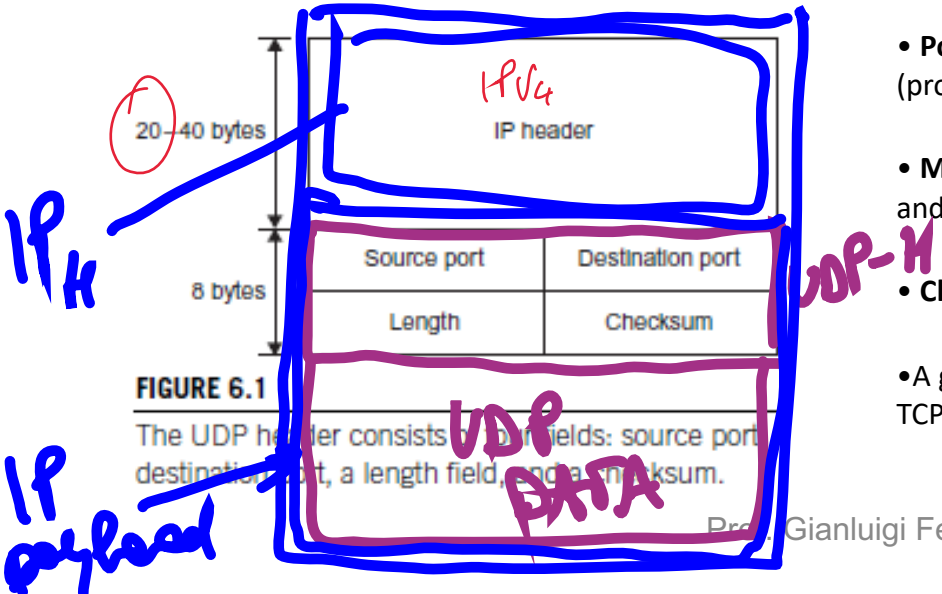


FIGURE 6.1

The UDP header consists of four fields: source port, destination port, a length field, and a checksum.

- **Port numbers** identify sending and receiving applications (processes). Maximum port number is $2^{16}-1= 65,535$
- **Message Length** is at least 8 bytes (I.e., Data field can be empty) and at most 65,535
- **Checksum** is for header (of UDP and some of the IP header fields)
- A globally unique address at the transport layer (for both UDP and TCP) is a tuple **<IP address, port number>**

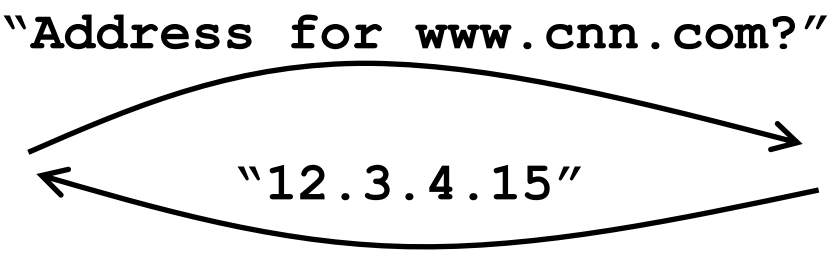
Why Would Anyone Use UDP?

- Finer control over what data is sent and when
  - As soon as an application process writes into the socket
  - … UDP will package the data and send the packet
- No delay for connection establishment
  - UDP just blasts away without any formal preliminaries
  - … which avoids introducing any unnecessary delays
- No connection state
  - No allocation of buffers, parameters, sequence #s, etc.
  - … making it easier to handle many active clients at once
- Small packet header overhead
  - UDP header is only eight-byte long
- **All excellent reasons for IoT scenarios**

Popular Applications That Use UDP

- Multimedia streaming
  - Retransmitting lost/corrupted packets is not worthwhile
  - By the time the packet is retransmitted, it's too late
  - E.g., telephone calls, video conferencing, gaming

- Simple query protocols like Domain Name System
  - Overhead of connection establishment is overkill
  - Easier to have application retransmit if needed



LEC. 3

Transmission Control Protocol (TCP)

- TCP provides a reliable byte stream on top of the best-effort packet service provided by the IP layer
- TCP hides the packet-oriented IP network beneath a virtual circuit abstraction, in which each virtual circuit is called a connection
- TCP provides a reliable stream transport service on top of the best-effort IP layer with three mechanisms:
  - Acknowledgments and retransmissions: All data sent with TCP are acknowledged by the receiver.
  - Sequence numbers: Every byte in the TCP byte stream is given a sequence number. The sequence numbers are used when matching acknowledgments with the corresponding data.
  - Sliding window (flow control): The receiver advertises how many bytes it is currently able to receive and the sender sends only as much data as the receiver can receive. As the receiver receives the data, it is able to receive more data.

An Analogy: Talking on a Cell Phone

- Alice and Bob on their cell phones
  - Both Alice and Bob are talking
- What if Alice couldn't understand Bob?
  - Bob asks Alice to repeat what she said
- What if Bob hasn't heard Alice for a while?
  - Is Alice just being quiet?
  - Or, have Bob and Alice lost reception?
  - How long should Bob just keep on talking?
  - Maybe Alice should periodically say "uh huh"
  - … or Bob should ask "Can you hear me now?" ☺

Some Take-Aways from the Example

- Acknowledgments from receiver
  - Positive: "okay" or "ACK"
  - Negative: "please repeat that" or "NACK"
- Timeout by the sender ("stop and wait")
  - Don't wait indefinitely without receiving some response
  - … whether a positive or a negative acknowledgment
- Retransmission by the sender
  - After receiving a "NACK" from the receiver
  - After receiving no feedback from the receiver

Segmented TCP Stream

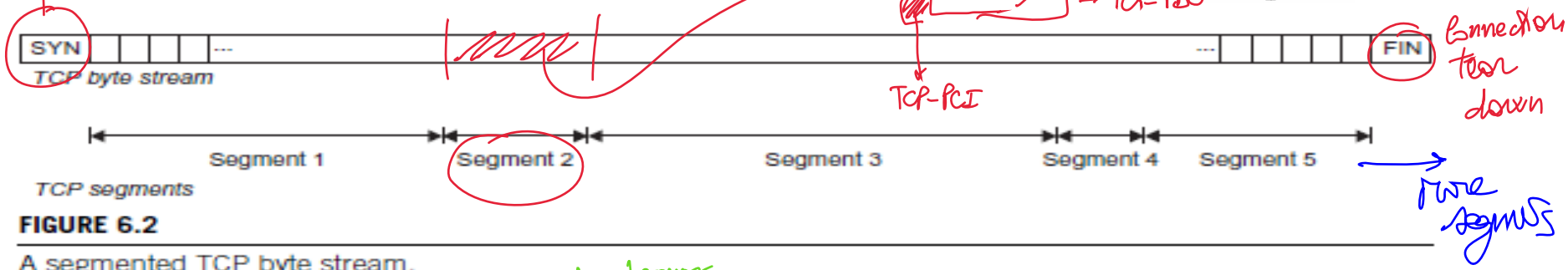


FIGURE 6.2

A segmented TCP byte stream.

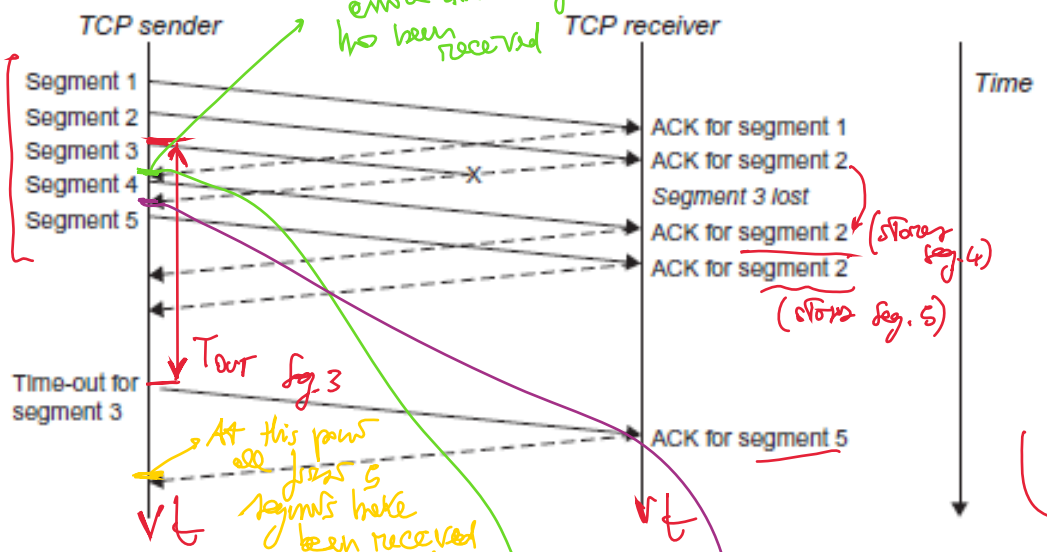


FIGURE 6.4

Loss of a TCP segment and the corresponding time-out.

(Fast retransmission: see p. 124)

Challenges of Reliable Data Transfer

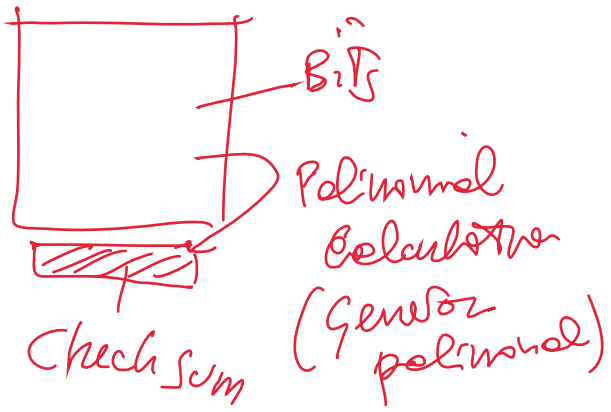
- Over a perfectly reliable channel (DREAM)
  - All of the data arrives in order, just as it was sent
  - Simple: sender sends data, and receiver receives data
- Over a channel with bit errors (ex. fiber optic channel)
  - All of the data arrives in order, but some bits corrupted
  - Receiver detects errors and says "please repeat that"
  - Sender retransmits the data that were corrupted
- Over a lossy channel with bit errors → (ex. wireless channel)
  - Some data are missing and some bits are corrupted
  - Receiver detects errors but cannot always detect loss
  - Sender must wait for acknowledgment ("ACK" or "OK")
  - … and retransmit data after some time if no ACK arrives

and no ACK will be transmitted

If a link is affected by fading, the packet will never reach the destination

TCP Support for Reliable Delivery

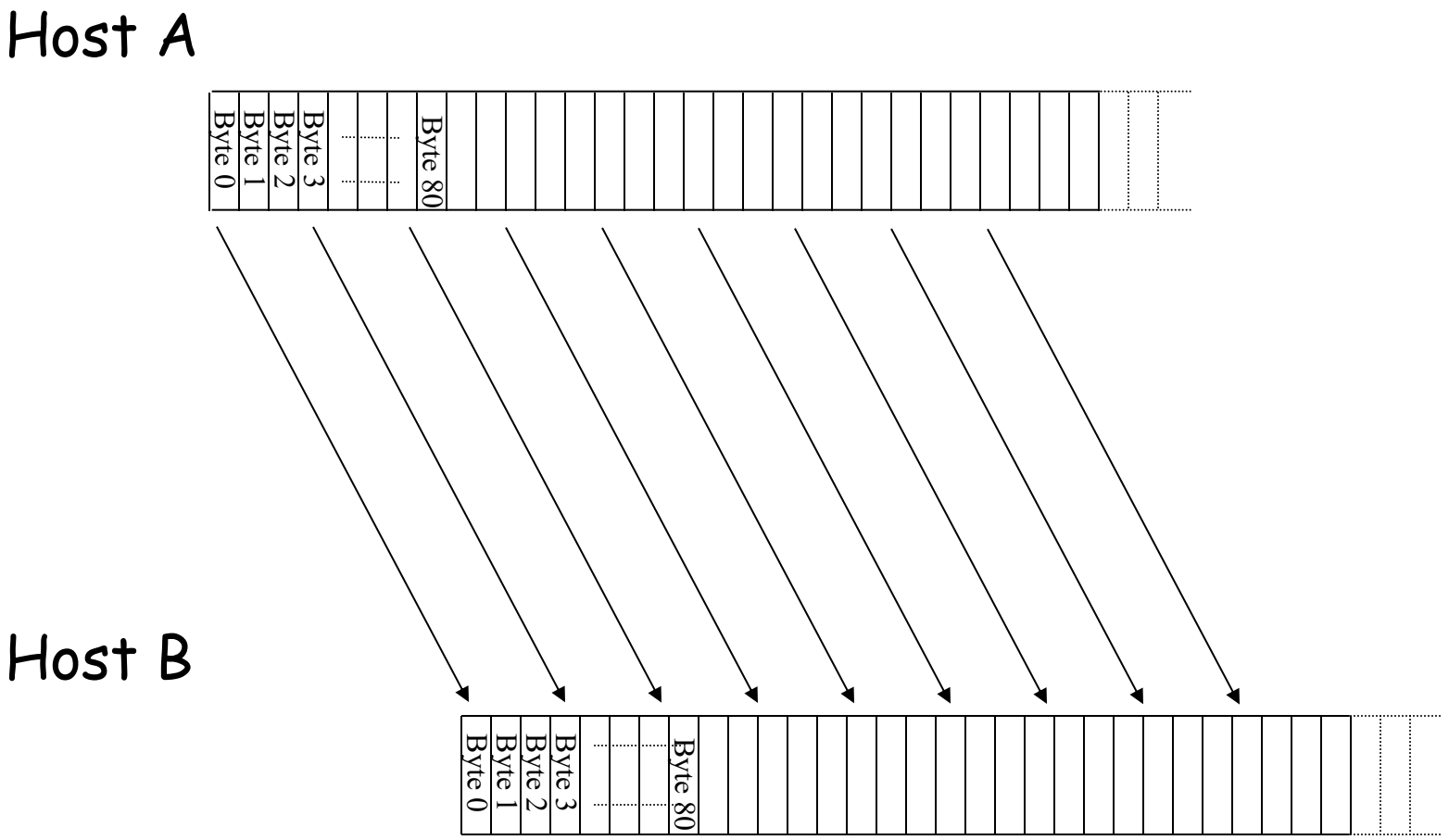
- Checksum
  - Used to detect corrupted data at the receiver
  - …leading the receiver to drop the packet
- Sequence numbers
  - Used to detect missing data
  - … and for putting the data back in order
- Retransmission
  - Sender retransmits lost or corrupted data
  - Timeout based on estimates of round-trip time
  - Fast retransmit algorithm for rapid retransmission



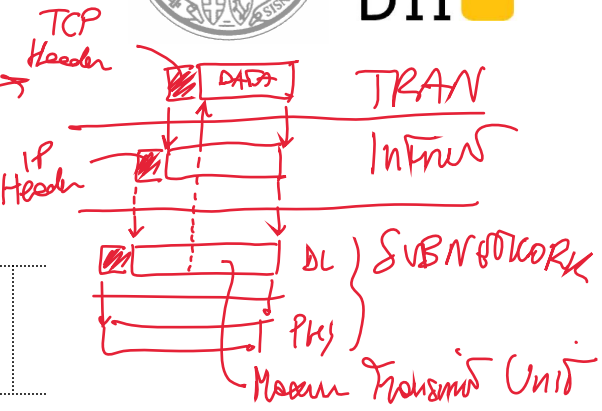
(No waiting for the time-out and immediate retransmission)

TCP Segments

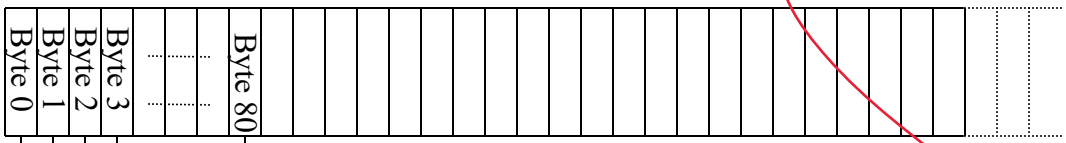
TCP "Stream of Bytes" Service



…Emulated Using TCP "Segments"



Host A



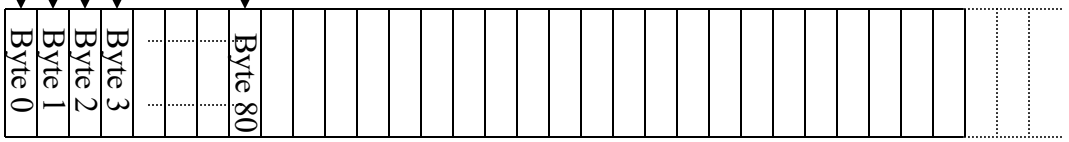
TCP Data

Segment sent when:

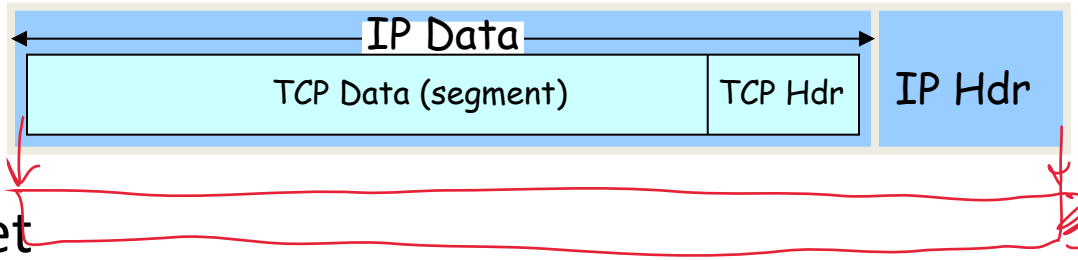
1. Segment full (Max Segment Size),
2. Not full, but times out, or
3. "Pushed" by application.

TCP Data

Host B

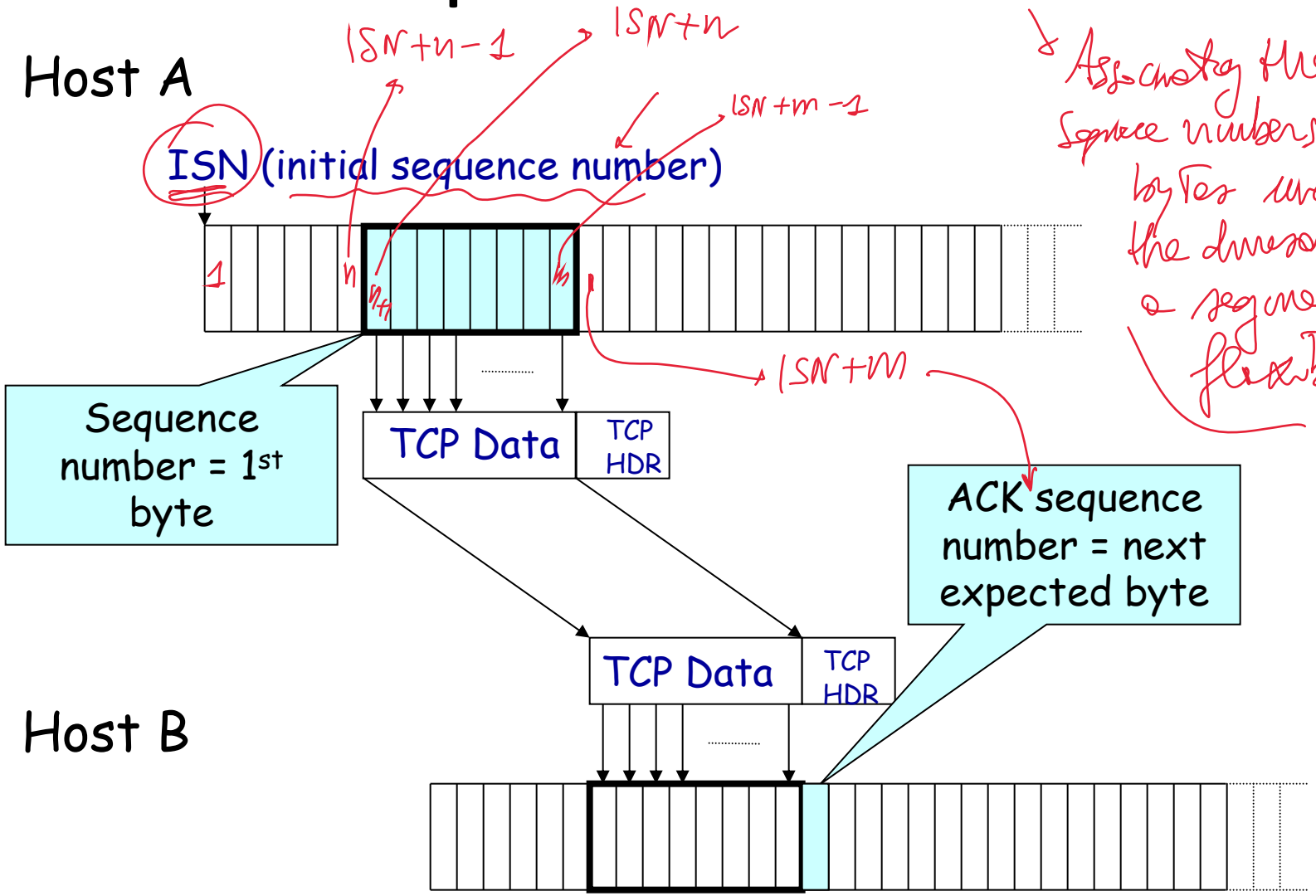


TCP Segment



- IP packet
  - No bigger than Maximum Transmission Unit (MTU)
  - E.g., up to 1500 bytes on an Ethernet
- TCP packet
  - IP packet with a TCP header and data inside
  - TCP header is typically 20 bytes long
- TCP segment
  - No more than Maximum Segment Size (MSS) bytes
  - E.g., up to 1460 consecutive bytes from the stream

Sequence Numbers



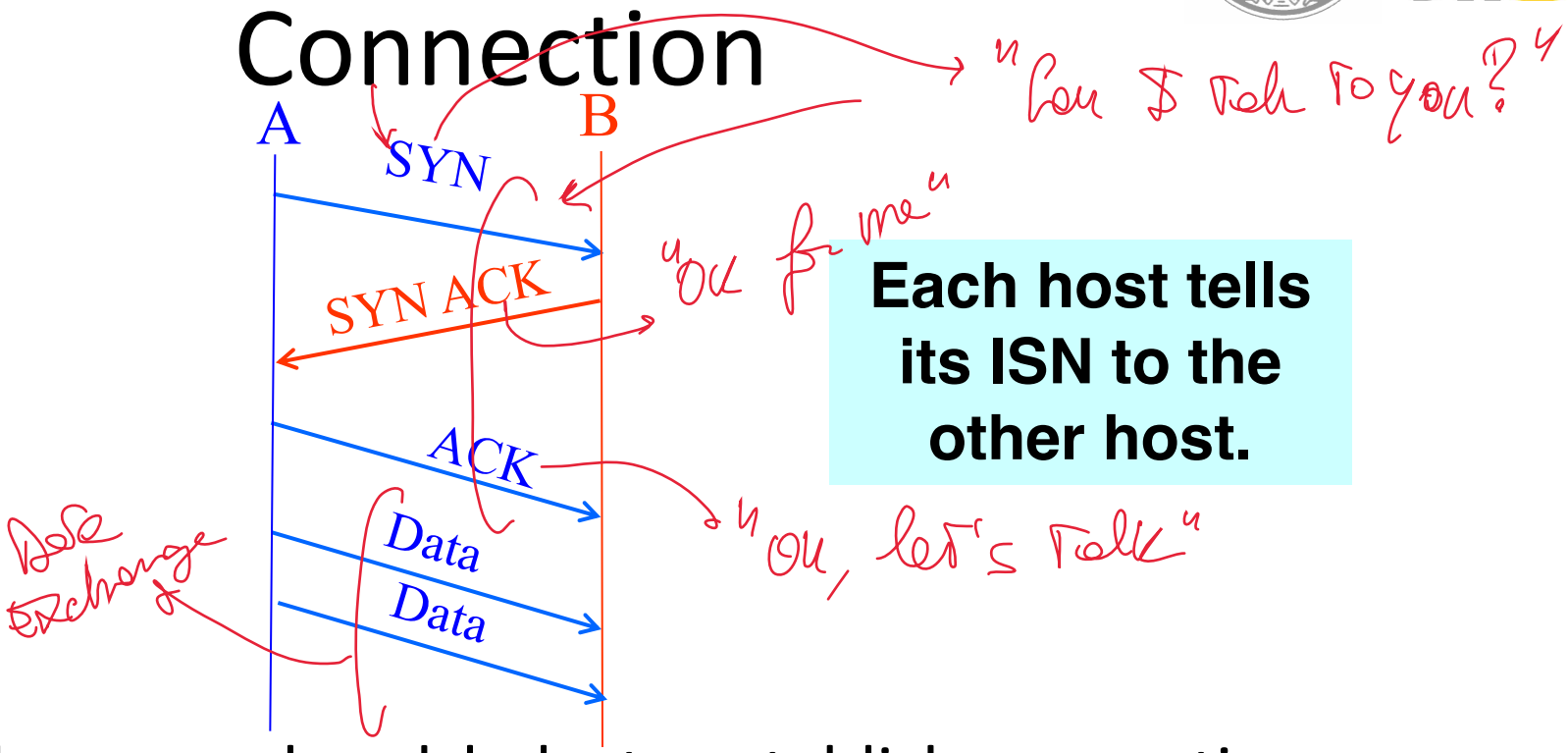
Initial Sequence Number (ISN)

- Sequence number for the very first byte
  - E.g., Why not a de facto ISN of 0?
- Practical issue
  - IP addresses and port #s uniquely identify a connection
  - Eventually, though, these port #s do get used again
  - … and there is a chance an old packet is still in flight
  - … and might be associated with the new connection
- So, TCP requires changing the ISN over time
  - Set from a 32-bit clock that ticks every 4 microseconds
  - … which only wraps around once every 4.55 hours!
- But, this means the hosts need to exchange ISNs

TCP Three-Way Handshake

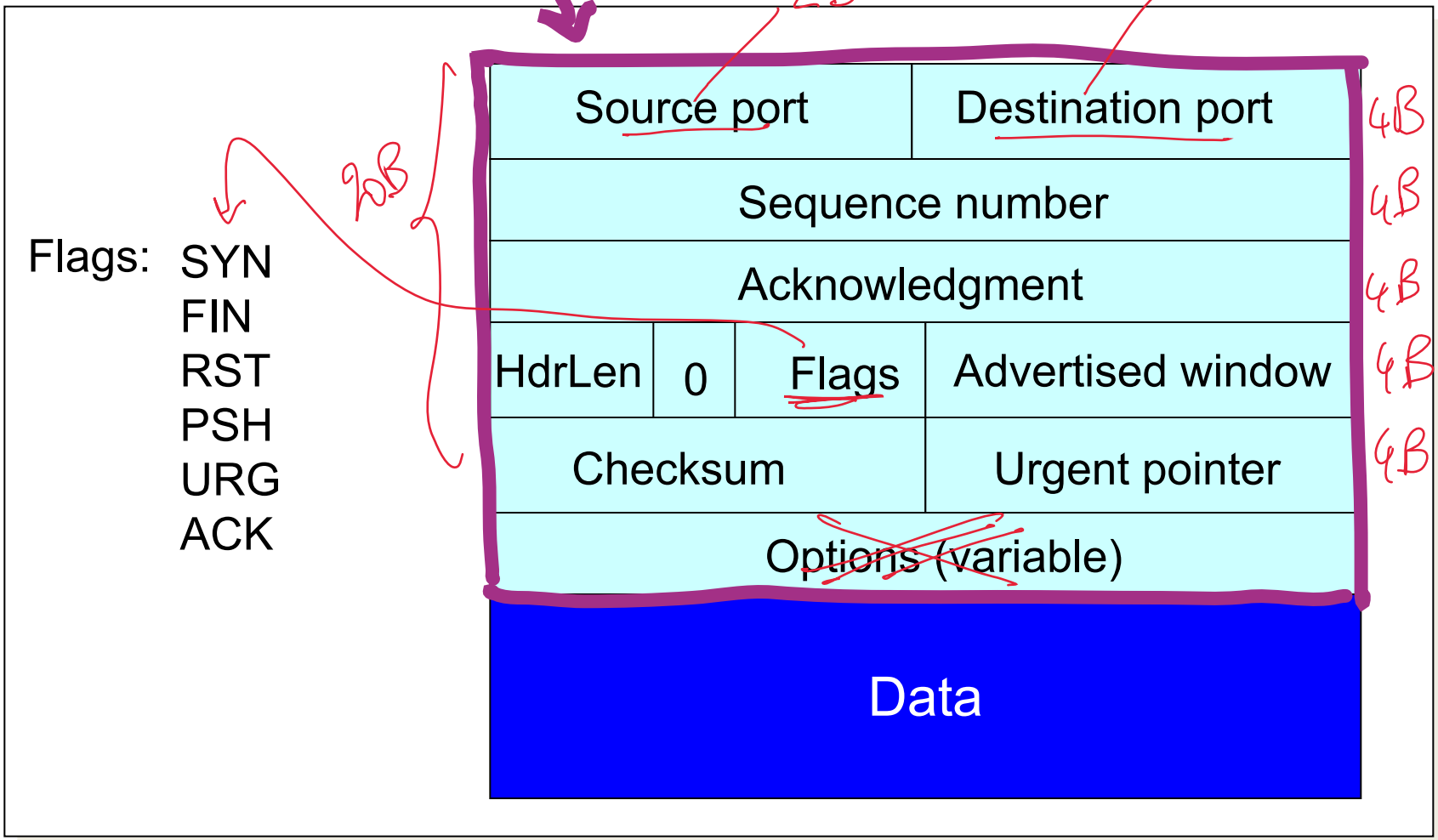
(Connection set-up)

Establishing a TCP Connection



- Three-way handshake to establish connection
  - Host A sends a **SYN** (open) to the host B
  - Host B returns a SYN acknowledgment (**SYN ACK**)
  - Host A sends an **ACK** to acknowledge the SYN ACK

TCP Header (20 bytes)



Step 1: A's Initial SYN Packet (A → B)

Flags: SYN
FIN
RST
PSH
URG
ACK

A's port			B's port
A's Initial Sequence Number			
Acknowledgment			
20 B	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

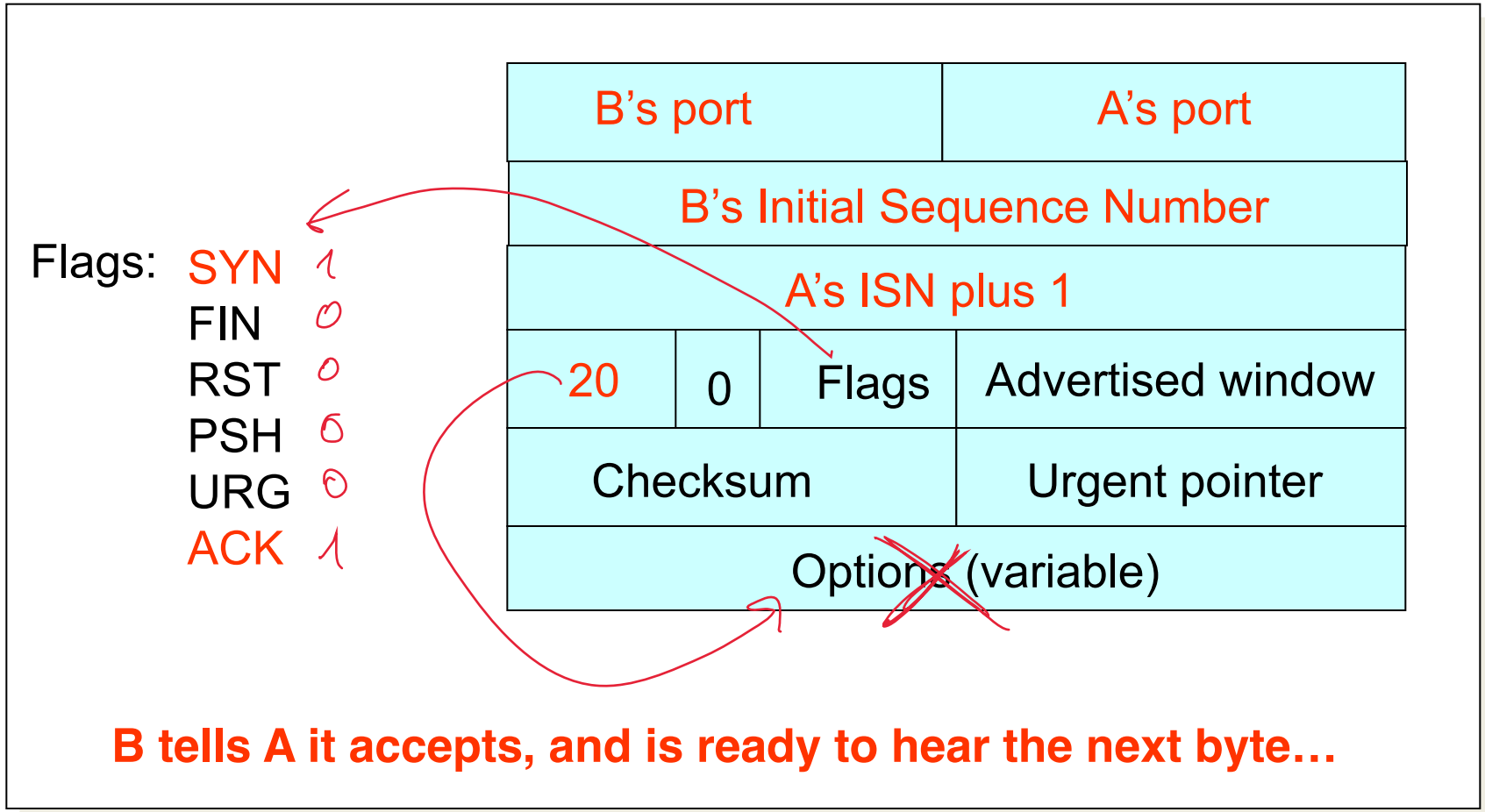
Header length

Flow window

A tells B it wants to open a connection…

(B → A)

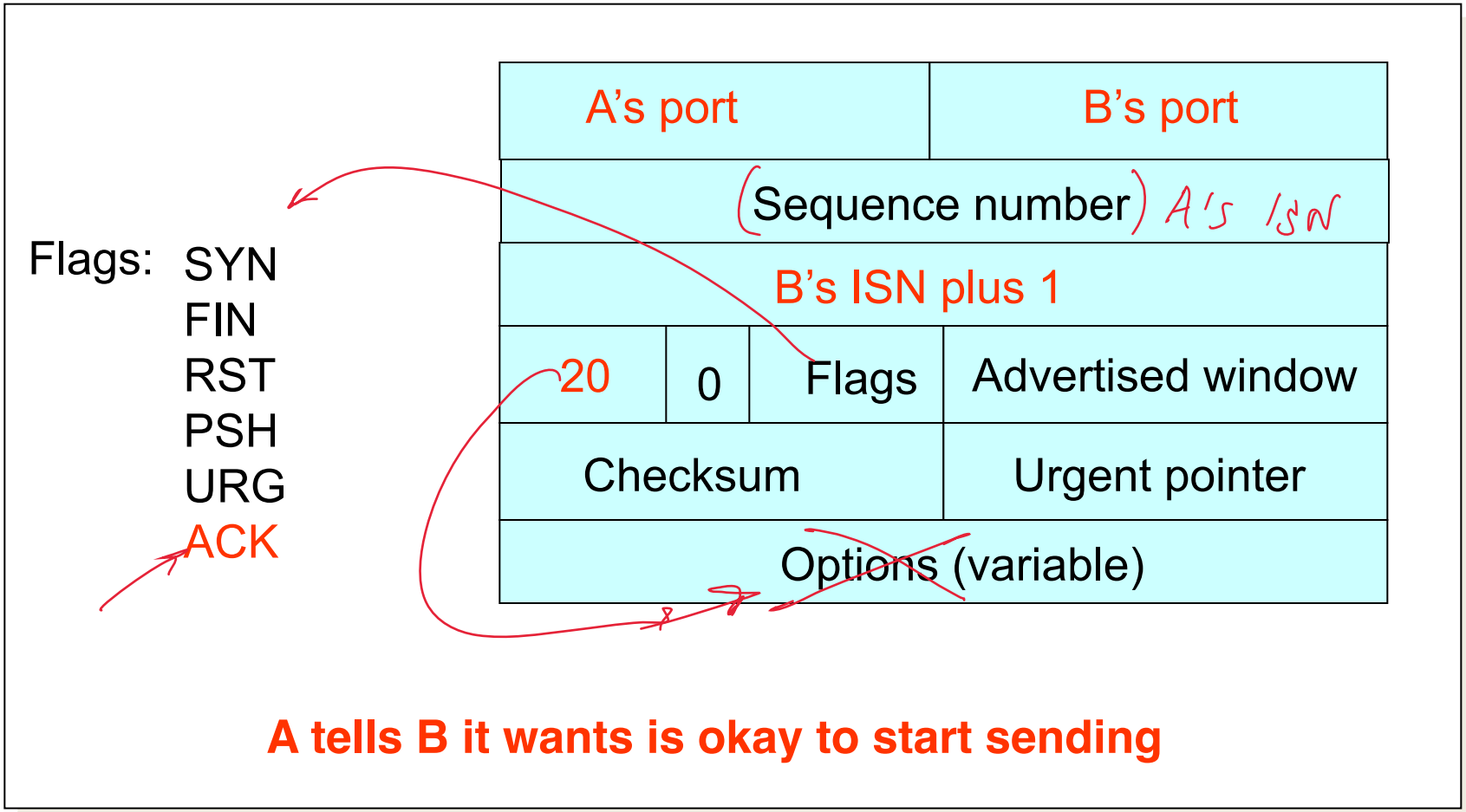
Step 2: B's SYN-ACK Packet



B tells A it accepts, and is ready to hear the next byte…

… upon receiving this packet, A can start sending data

Step 3: A's ACK of the SYN-ACK (A → B)



A tells B it wants is okay to start sending

… upon receiving this packet, B can start sending data

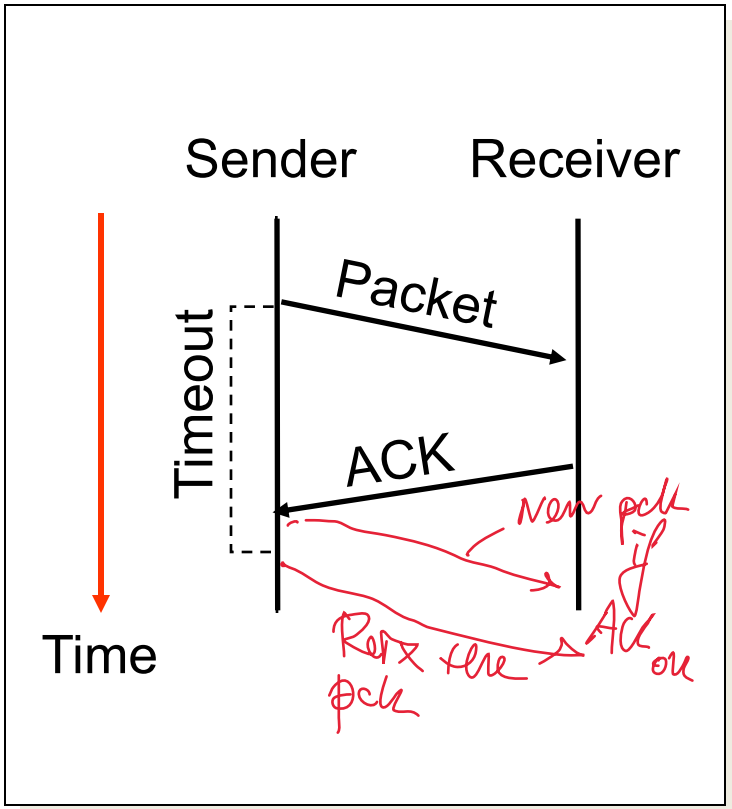
What if the SYN Packet Gets Lost?

- Suppose the SYN packet gets lost
  - Packet is lost inside the network, or
  - Server rejects the packet (e.g., listen queue is full)
- Eventually, no SYN-ACK arrives
  - Sender sets a timer and wait for the SYN-ACK
  - … and retransmits the SYN-ACK if needed
- How should the TCP sender set the timer?
  - Sender has no idea how far away the receiver is
  - Hard to guess a reasonable length of time to wait
  - Some TCPs use a default of 3 or 6 seconds

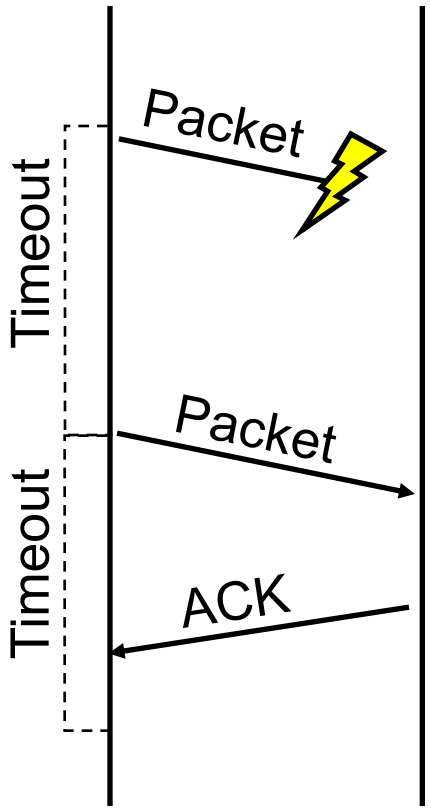
TCP Retransmissions

Automatic Repeat reQuest (ARQ)

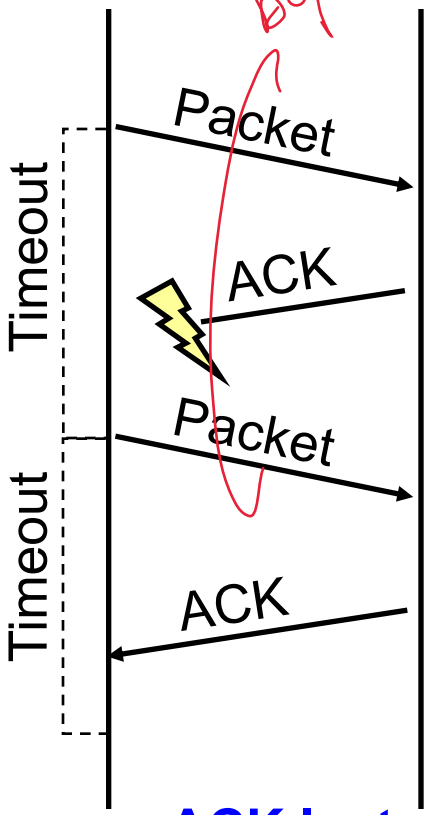
- Automatic Repeat Request
  - Receiver sends acknowledgment (ACK) when it receives packet
  - Sender waits for ACK and timeouts if it does not arrive within some time period
- Simplest ARQ protocol
  - Stop and wait
  - Send a packet, stop and wait until ACK arrives



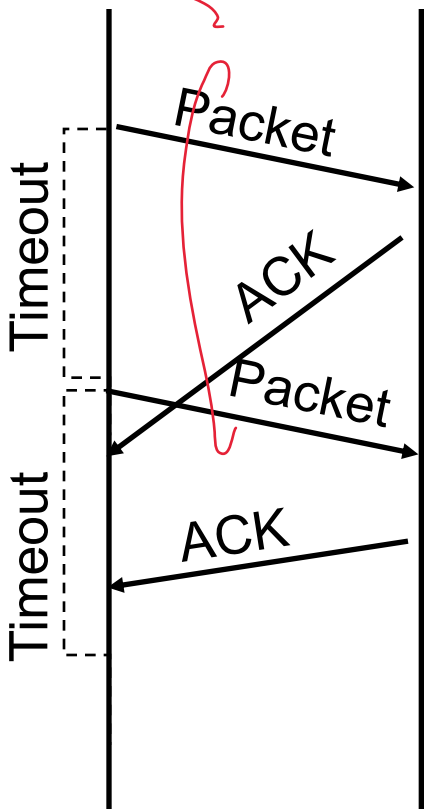
Reasons for Retransmission



Packet lost

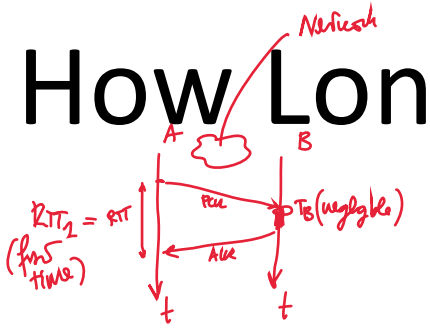


ACK lost
DUPLICATE PACKET



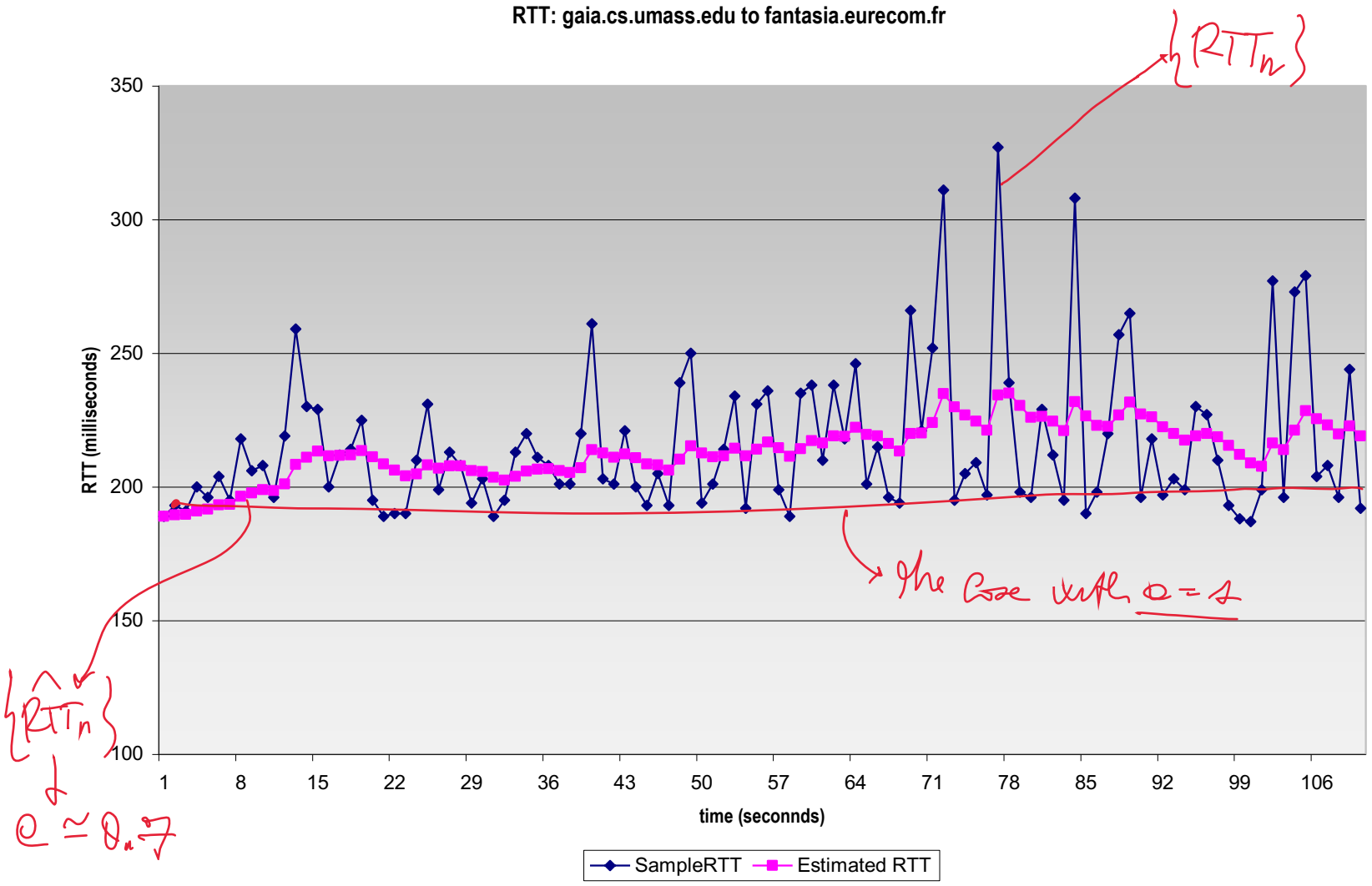
Early timeout
DUPLICATE PACKETS

How Long Should Sender Wait?



- Sender sets a timeout to wait for an ACK
  - Too short: wasted retransmissions
  - Too long: excessive delays when packet lost
- TCP sets timeout as a function of the RTT (Round Trip Time)
  - Expect ACK to arrive after an RTT
  - … plus a fudge factor to account for queuing
- But, how does the sender know the RTT?
  - Can estimate the RTT by watching the ACKs
  - Smooth estimate: keep a running average of the RTT
    - EstimatedRTT = a * EstimatedRTT + (1 –a) * SampleRTT
  - Compute timeout: TimeOut = 2 * EstimatedRTT

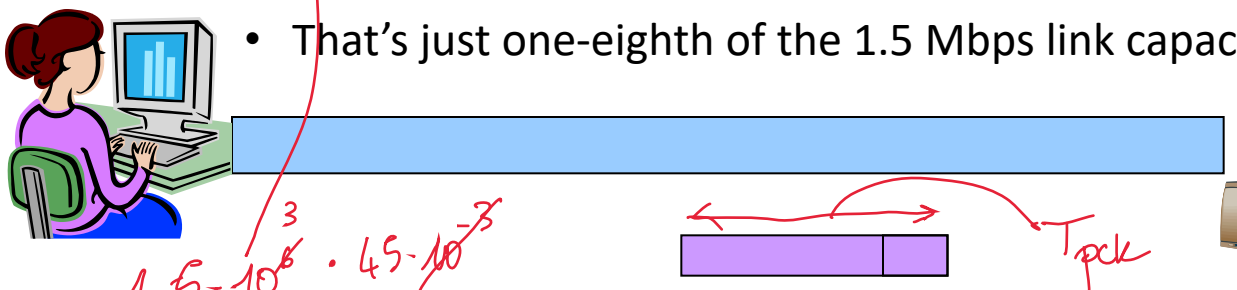
Example RTT Estimation



TCP Sliding Window

Motivation for Sliding Window

- Stop-and-wait is inefficient
  - Only one TCP segment is "in flight" at a time
  - Especially bad when delay-bandwidth product is high
- Numerical example
  - 1.5 Mbps link with a 45 msec round-trip time (RTT)
    - Delay-bandwidth product is 67.5 Kbits (or 8 KBytes)
  - But, sender can send at most one packet per RTT
    - Assuming a segment size of 1 KB (8 Kbits)
    - … leads to 8 Kbits/segment / 45 msec/segment ➔ 182 Kbps
    - That's just one-eighth of the 1.5 Mbps link capacity



Sliding Window

- Allow a larger amount of data "in flight"
  - Allow sender to get ahead of the receiver
  - … though not *too far* ahead